分布式数据库系统 往年试题参考解析

2018、2019、2020、2023 年试题，以及 2004–2011 年的概念题

题目按类型分成七部分。每类题先写做法，然后用问答讲清每一步的理由，灰底双线框是参考答卷，虚线框是容易出错的地方。“课件 3-21”指打印版 PPT 每页右上角的编号。

按题型查找

题目里出现	题型	本文	课件
“按需求分片”“分片定义”“分片树”“分片类型”	水平 / 垂直 / 导出 / 混合	§ 1.1 p.2	3-21–55
“分配设计”“分配类型”	全分割 / 全复制 / 部分复制	§ 1.1 p.2	3-57–63
“写出关系代数和 SQL”“全局查询树”	查询 → 关系代数 → 查询树	§ 1.2 p.3	5-21–23, 48
“全局优化，写出中间过程”	选择、投影下推	§ 1.2 p.3	5-49–55
“分片优化”“片段查询树”	本地化与剪枝	§ 1.2 p.3	5-57–69
“内存存放 5 块”“每秒读写 25 块”	I/O 代价	§ 2.1 p.10	5-11–18
“S1、S2、S3，选传输代价最小的执行场地”	选择执行场地	§ 2.2 p.11	6-5–7
“全连接”“优化的查询策略”“通信代价”，给了 Card、Length、Val	直接连接与半连接	§ 2.3 p.11	6-15–39
“分解为子事务”“进程代理模型”	子事务与伪代码	§ 3.1 p.14	7-19–26
“集中式两阶段提交”“协调者和参与者”	2PC 流程与日志	§ 3.2 p.17	7-29–34, 47
“某场地出故障”“丢失 commit 命令”“转账没有成功”	2PC 的故障处理	§ 3.3 p.18	8-42–55
“冲突对”“局部可串行”“全局可串行”“构建历程”	可串行化	§ 4.1 p.21	9-10–18
“两阶段封锁协议”	2PL	§ 4.5 p.23	9-24–33
日志 $B_1\ D_1^1\ C_1\ K\ \ldots$，“undo”“redo”“检查点”	故障恢复	§ 5 p.24	8-13–40
“主副本”“同步复制协议”“如何保证一致性”	主从同步复制	§ 6.1 p.26	10-11–28
“Paxos、反熵、NWR 的异同”	一致性协议比较	§ 6.2 p.27	10-38–54
ACID、集中式与分布式的区别、区块链、学习收获	概念题	§ 7 p.28	7-9–14, 1-52

记号

$\sigma_{\mathrm{F}}(R)$	选择，留下满足 F 的元组	$R \bowtie S$	连接，课件写作 $R \infty S$
$\Pi_{\mathrm{A}}(R)$	投影，留下属性 A	$R \ltimes S$	半连接，课件写作 $R \propto S$，等于 $\Pi_{\mathrm{Attr(R)}}(R \bowtie S)$
h、v	分片树的边：水平、垂直	$\cup$	并。水平片段用 $\cup$ 重构，垂直片段用 $\bowtie$ 重构
$\mathrm{Card}(R)$	元组数	$\mathrm{Length}(R)$	一个元组的字节数
$\mathrm{Size}(R)$	$\mathrm{Card}(R) \times \mathrm{Length}(R)$	$\mathrm{Val}(R, A)$	属性 A 不同取值的个数
ρ	选择度，满足条件的元组所占比例	C_0, C_1	一次传输的启动代价，单位数据传输代价
B_i, C_i, A_i	事务 T_i 开始、提交、废弃	D_i^j	T_i 的第 j 个数据操作；K 是检查点
$R_i(x), W_i(x)$	T_i 读、写数据项 x	$T_i < T_j$	在等价的串行顺序中 T_i 在 T_j 前

计算题和设计题的答题顺序一般是：先写假设（数据均匀分布、单位、属性取值的含义），再写依据的定义或公式，然后分步计算或作图，每步注明用了哪条规则，最后写结论。题目要求“写出中间过程”时，每次等价变换都要单独写一行或画一棵树。

第1部分 分片设计、分配设计与查询优化

每年第 1 题都是这一类，约 20 分。常见的五问是：分片定义、分片树和分片类型；分配设计和分配类型；SQL、关系代数和全局查询树；全局优化；分片优化。

1.1 分片设计与分配设计

分片与分配设计的步骤

1. 把每条需求写成“哪个关系的哪些数据存在哪个场地”。
2. 确定分片类型。按元组划分是水平分片，例如“各单位的员工”“涉密项目”；按属性划分是垂直分片，例如“属性 {...} 存在某场地”。划分元组的条件属性在另一个关系里时（“存在负责人所在场地”“存在发生借贷的分行”），用导出水平分片，写成半连接 $\ltimes$。先垂直再水平是混合分片。
3. 需求里有“其中满足 C 的在某场地保存副本”时，把 C 与 $\neg C$ 也作为分片条件再切一次。
4. 写分片定义。每个片段写成 $\sigma_{\text{条件}}(R)$、$\Pi_{\text{属性}}(R)$ 或 $R \ltimes S'$；同一层的条件互斥，并且覆盖全部取值。
5. 用一句话说明满足三条原则：完备性（片段的并等于原关系），可重构性（用 $\cup$ 或 $\bowtie$ 能还原），不相交性（水平片段两两相交为空，垂直片段之间只共享主键）。
6. 画分片树。根是全局关系，叶子是最终片段，边上标 h 或 v [课件3-54]。
7. 分配设计：列出每个场地存放的片段。每个片段只存一处是全分割，每个片段在每个场地都有是全复制，其余是部分复制 [课件3-57]。

问 分片和分配有什么区别?

答 分片是逻辑划分，把全局关系分成片段；分配是物理放置，决定片段存在哪个场地 [课件3-13]。片段是分配的最小单位，要放就整个放到某个场地，不能只放一部分。同一个片段存在多个场地，就是复制。

问 “金额 $\geq$100 万的项目在总部保存副本”，为什么要因此多切一次?

答 副本只能以整个片段为单位。如果某个分支机构的项目都在片段 P_1 里，而总部只要其中金额大的部分，就没法复制。把 P_1 分成 P_{11}（$\geq$100）和 P_{12}（<100）以后，P_{11} 放两个场地，P_{12} 只放一个场地。题目里每出现一条副本要求，一般就要多一个分片条件。

问 什么时候用导出水平分片?

答 划分依据的属性在别的关系里时。例如“企业信息存在与它有借贷关系的分行”：企业表里没有分行号，分行号 Bid 在贷款表里，所以用半连接把贷款表的划分带过来：$B_i = \text{Loanbusiness} \ltimes_{\text{Lid}} \sigma_{\text{Bid='Ai'}}(\text{Loaninfo})$，即在 A_i 有贷款的企业 [课件3-36–38]。

问 垂直分片为什么每个片段都要带主键?

答 垂直片段靠连接重构（$R = R_1 \bowtie R_2$），连接需要公共属性，这个公共属性就是主键。课件的条件是 $\text{Attr}(R_i) \cap \text{Attr}(R_j) = \text{PK}(R)$ [课件3-40]：主键以外的属性只能出现在一个片段里。

问 分片条件怎么写?

答 同一层的条件两两矛盾，保证不相交；合起来恒为真，保证完备。成对写 C 和 $\neg C$ 不会出错。按 Bid=A1/A2/A3 枚举时，要求 Bid 只有这几个取值，可以写一句“设 Bid 只取 A0–A3”。

注意

- 混合分片第二层的对象是垂直片段，要写 $\sigma_{\text{Dno=0}}(\text{Student}_1)$，不写 $\sigma_{\text{Dno=0}}(\text{Student})$。
- 分配类型按整个方案判断：有片段被复制，又不是所有片段都在所有场地，就是部分复制。
- 不分片的关系也要写出来，例如“DEPT 不分片，存放在 S0”。

1.2 查询优化：全局优化和分片优化

查询优化的步骤

1. SQL：FROM 写涉及的全局关系；WHERE 写连接条件和选择条件；SELECT 写要输出的属性。
2. 关系代数写成 $\Pi_{\text{输出属性}}\big(\sigma_{\text{全部选择条件}}(R \bowtie S \bowtie \cdots)\big)$。
3. 初始全局查询树：根是 Π，下面是 σ，再下面是 $\bowtie$，叶子是全局关系 [课件5-48]。
4. 全局优化 [课件5-49–55]，每一步注明依据：(a) 用选择串接律拆开 $\sigma_{F_1 \wedge F_2}$；(b) 用选择与连接的分配律，把每个条件移到只含其属性的关系上方；(c) 用投影串接律和投影与连接的分配律，在每个叶子上方加 Π，只留输出属性和连接属性；(d) 画出优化后的全局查询树。
5. 分片优化 [课件5-57–69]：(a) 数据本地化，把全局关系换成分片树，h 换成 $\cup$，v 换成 $\bowtie$；(b) 准则 1：把 σ 移到每个片段上，与片段条件矛盾的置为空并删去；(c) 准则 3：把 $\bowtie$ 移到 $\cup$ 下面，$(A \cup B) \bowtie C = (A \bowtie C) \cup (B \bowtie C)$；准则 2：连接条件不可能满足的分支为空；(d) 准则 4：删去对查询没有贡献的垂直片段；(e) 画出最终的片段查询树，注明各片段所在场地。

全局优化常用的等价变换规则 [课件5-49–50]（写中间过程时在括号里注明）

选择串接律	$\sigma_{F_1}(\sigma_{F_2}(R)) = \sigma_{F_1 \wedge F_2}(R)$
选择与连接（笛卡尔积）的分配律	若 F 只涉及 R 的属性：$\sigma_{\mathrm{F}}(R \bowtie S) = \sigma_{\mathrm{F}}(R) \bowtie S$
选择与并的分配律	$\sigma_{\mathrm{F}}(R \cup S) = \sigma_{\mathrm{F}}(R) \cup \sigma_{\mathrm{F}}(S)$，分片优化中把 σ 移进 $\cup$ 用它
投影串接律	$A \subseteq B$ 时 $\Pi_{\mathrm{A}}(\Pi_{\mathrm{B}}(R)) = \Pi_{\mathrm{A}}(R)$
投影与连接的分配律	$\Pi_{A \cup B}(R \bowtie S) = \Pi_{\mathrm{A}}(R) \bowtie \Pi_{\mathrm{B}}(S)$，$A$、$B$ 要包含连接属性

问 为什么先把选择和投影往下移?

答 课件的原则是先做能缩小中间结果的运算 [课件5-51]。σ、Π 的代价是 $O(n)$，连接是 $O(n \log n)$，笛卡尔积是 $O(n^2)$。先用 σ 减少元组、用 Π 去掉用不到的属性，后面的连接就便宜得多。2023 年第三题里同一个查询的三种写法，最慢的约 22 小时，最快的 8 秒。

问 投影下移时，为什么不能只保留 SELECT 里的属性?

答 上面还要做连接。把 Π_{Sname} 移到 Student 上方时，如果连 Sno 也去掉，Student 就无法再和 SC 按 Sno 连接。每个叶子上方保留的是输出属性加上层连接要用的属性。

问 条件 Dno=2 只写在 Student 上，DEPT 上也能加吗?

答 能。WHERE 里有 Student.Dno=DEPT.Dno，所以 Student.Dno=2 也意味着 DEPT.Dno=2。两边都先筛选，连接的两个输入都变小。2023 年的参考答案也是两边都加。

问 全局优化和分片优化有什么区别?

答 全局优化只用代数变换规则，处理的是全局关系。分片优化把全局关系换成片段以后，可以根据片段的定义条件判断哪些片段查不出结果，直接删掉。最后往往只剩一两个片段，有时整个查询在一个场地就能完成。

问 怎样判断片段和查询条件矛盾?

答 把片段条件和查询条件做合取，看是否恒假。例如片段 E_{21} 的条件是 Bid='A2'，查询要求 Bid='A1'，合取为假，所以 $\sigma_{\text{Bid='A1'}}(E_{21}) = \varnothing$。答题时可以列一张表，写出片段、片段条件、查询条件和是否矛盾，作为中间过程。

问 导出片段之间做连接时，怎样删分支?

答 用准则 2。B_i 是在 A_i 有贷款的企业，L_{1j} 是 A1 的贷款。如果每个企业只在一个分行贷款，$L_{1j} \bowtie B_2$ 和 $L_{1j} \bowtie B_3$ 都为空，只保留 $L_{1j} \bowtie B_1$。

问 什么情况下可以删垂直片段?

答 一个垂直片段除主键外的属性既不输出、也不出现在条件里，就可以删（准则 4）。例如只查 ENO、ENAME、BIRTH，这三个属性都在 E_1 里，E_2 的子树可以整个删去 [课件5-68–69]。如果输出要 Sname、条件要 Dno，而这两个属性分在两个垂直片段里，两个片段都要保留，用 Sno 连接。

注意

- 题目要求写中间过程时，至少画三棵树：初始树、全局优化后的树、分片优化后的树。分片优化可以再加一棵本地化后的树或一张剪枝表。
- 片段条件已经蕴含的选择可以去掉，例如 $\sigma_{\text{Dno=2}}(\text{Student}_{12}) = \text{Student}_{12}$，保留也不算错。
- 片段有副本时（如 E_{11} 同时在 A1 和 A0），分片优化不需要选副本。选副本属于存取优化，可以补一句“选 A1 上的副本，本地执行”。

1.3 2019 年第 1 题（20 分）

原题

某银行由总行和三个分行组成，三个分行编号分别为 A_1、A_2 和 A_3，分别执行具体的放贷业务，总行没有放贷业务（编号 A_0）。现对贷款企业进行资产管理，涉及到的全局关系模式如下：

Branchers(Bid, Bname, Addrs, Mgrno)：分行基本信息表。Bid 为分行编号，Bname 为分行名称，Adrs 是分行地址，Mgrno 是分行负责人员工编号。

Employee(Eid, Ename, Etitle, Bid)：银行员工基本信息表。Eid 为员工编号，Ename 为员工姓名，Etitle 为员工职务，Bid 是员工所属的分行编号。

Loanbusiness(Lid, Lname, Lcapital, Ladr, Lpid)：贷款企业信息表。Lid 为企业编号，Lname 为企业名称，Lcapital 为企业注册资本，Ladr 为企业地址，Lpid 为企业负责人身份证号码。

Loaninfo(Bid, Lid, Eid, Lamount, Ldate, Lquota)：贷款信息表。Bid 为分行编号，Lid 为借贷企业编号，Eid 为银行贷款员编号，Lamount 为贷款金额，Ldate 为放贷日期，Lquota 为贷款期限。

数据管理的基本需求是：(1) 所有分行的基本信息都保存在总行场地；(2) 总行和各分行保存各自的员工信息，职务为“贷款员”(Etitle='Loan operator') 的员工信息在总行保存副本；(3) 各分行保存本分行员工发放的贷款信息，但单笔贷款金额≥1000 万元的贷款信息在总行保存副本；(4) 企业信息保存在发生借贷关系的分行。

1) 按要求进行分片设计，要求写出分片定义，画出分片树，并指出分片类型；

2) 按要求完成分配设计，指出分配类型；

3) 查询分行 A_1 贷款员张龙向沈阳地区的企业发放的单笔贷款金额在 500 万元及以下的贷款信息。要求查询结果包括贷款企业名称、企业地址、贷款金额、放贷日、贷款期限。请写出查询的关系代数形式和 SQL 语句形式，并画出全局查询树；

4) 对 3) 题的查询进行全局优化，要求写出中间过程；

5) 在全局优化的基础上进行片段优化，要求写出中间过程。

问 四条需求分别涉及哪个关系？

答 (1) 只涉及 Branchers，整体放在 A_0，不分片。(2) 涉及 Employee：按 Bid 水平分片，为了副本再按是否为贷款员切一次。(3) 涉及 Loaninfo：按 Bid 水平分片，为了副本再按 Lamount≥1000 切一次。(4) 涉及 Loanbusiness：划分依据“与哪个分行借贷”在 Loaninfo 里，所以是导出水平分片。

问 总行员工 E_0 要不要再按贷款员切？

答 不用。E_0 本来就存在 A_0，“在总行存副本”对它没有意义；总行也没有放贷业务，没有贷款员。

问 一个企业同时在 A1 和 A2 贷款，导出片段 B_1、B_2 不就相交了吗？

答 会相交，这时按需求它确实要存在两个分行，相当于有副本。答卷里写一个假设：“设每个企业只与一个分行发生借贷关系，则 B_1, B_2, B_3 两两不相交”。另外，没贷过款的企业不在任何 B_i 里，所以还要假设每个企业至少有一笔贷款，才满足完备性。

参考答卷：1) 分片定义、分片树、分片类型

Branchers：不分片，整体存放在 A_0。

Employee：水平分片，分片属性 Bid、Etitle，记 LO = 'Loan operator'。 $E_0 = \sigma_{\text{Bid='A0'}}(\text{Employee})$；对 $i = 1, 2, 3$：

$E_{i1} = \sigma_{\text{Bid='Ai'} \wedge \text{Etitle=LO}}(\text{Employee})$， $E_{i2} = \sigma_{\text{Bid='Ai'} \wedge \text{Etitle}\neq\text{LO}}(\text{Employee})$。

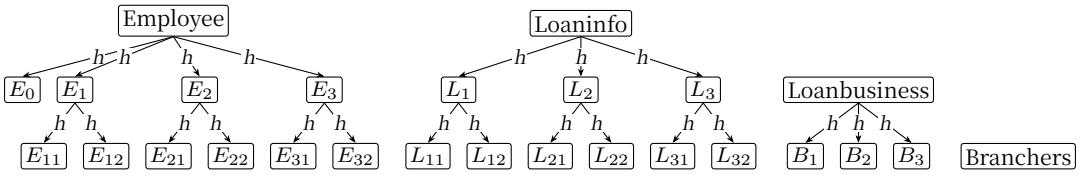
Loaninfo：水平分片，分片属性 Bid、Lamount（单位万元）。对 $i = 1, 2, 3$：

$L_{i1} = \sigma_{\text{Bid='Ai'} \wedge \text{Lamount}\geq 1000}(\text{Loaninfo})$， $L_{i2} = \sigma_{\text{Bid='Ai'} \wedge \text{Lamount}<1000}(\text{Loaninfo})$。

Loanbusiness：由 Loaninfo 的 Bid 导出的水平分片。对 $i = 1, 2, 3$：

$B_i = \text{Loanbusiness} \ltimes_{\text{Lid}} \sigma_{\text{Bid='Ai'}}(\text{Loaninfo}) = \text{Loanbusiness} \ltimes_{\text{Lid}} (L_{i1} \cup L_{i2})$。

假设每个企业至少有一笔贷款，且只与一个分行有借贷；Bid 只取 A0–A3。各层条件互斥且覆盖全部取值，满足完备性、可重构性（用 $\cup$ 重构）和不相交性。



参考答卷：2) 分配设计与分配类型

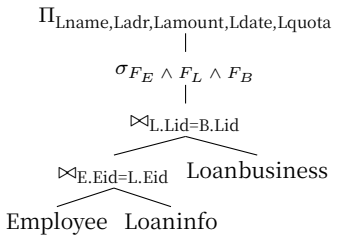
A_0（总行）： Branchers，E_0，E_{11}, E_{21}, E_{31}（副本），L_{11}, L_{21}, L_{31}（副本）

A_i（$i = 1, 2, 3$）： E_{i1}, E_{i2}，L_{i1}, L_{i2}，B_i

E_{i1}、L_{i1} 存放在两个场地，其余片段只存放在一个场地，是部分复制分配。

参考答卷：3) SQL、关系代数、全局查询树

```
SELECT Lname, Ladr, Lamount, Ldate, Lquota
FROM  Employee E, Loaninfo L, Loanbusiness B
WHERE E.Eid=L.Eid AND L.Lid=B.Lid
AND E.Ename='张龙' AND E.Bid='A1'
AND E.Etitle='Loan operator'
AND L.Bid='A1' AND L.Lamount<=500
AND B.Ladr='沈阳';
```



关系代数：$\Pi_{\text{Lname,Ladr,Lamount,Ldate,Lquota}}\big(\sigma_{F_E \wedge F_L \wedge F_B}((\text{Employee} \bowtie_{\text{Eid}} \text{Loaninfo}) \bowtie_{\text{Lid}} \text{Loanbusiness})\big)$，其中

F_E：Ename='张龙' $\wedge$ Etitle='Loan operator' $\wedge$ Bid='A1'

F_L：Bid='A1' $\wedge$ Lamount ≤ 500

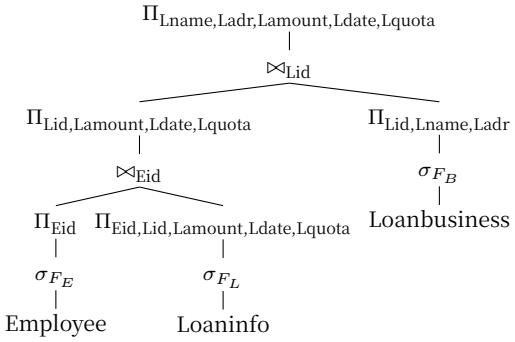
F_B：Ladr='沈阳'

注意

Employee 和 Loaninfo 都有 Bid，写成自然连接会把 Bid 也当作连接属性，所以要写带条件的连接 $\bowtie_{\text{Eid}}$。条件 L.Bid='A1' 来自题意（A1 发放的贷款），写出来以后，分片优化能删掉 A2、A3 的贷款片段。

参考答卷：4) 全局优化（中间过程）

步骤 1（选择串接律）：$\sigma_{F_E \wedge F_L \wedge F_B} = \sigma_{F_E}\,\sigma_{F_L}\,\sigma_{F_B}$。
步骤 2（选择与连接的分配律）：F_E 只含 Employee 的属性，移到 Employee 上方；F_L 移到 Loaninfo 上方；F_B 移到 Loanbusiness 上方。
步骤 3（投影串接律、投影与连接的分配律）：每个叶子上方只保留输出属性和连接属性。Employee 留 {Eid}；Loaninfo 留 {Eid, Lid, Lamount, Ldate, Lquota}；Loanbusiness 留 {Lid, Lname, Ladr}；第一次连接后留 {Lid, Lamount, Ldate, Lquota}。

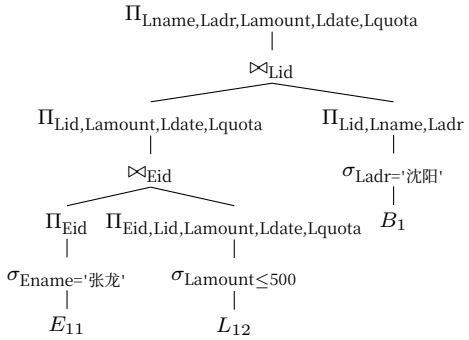


参考答卷：5) 分片优化（中间过程）

步骤 1（数据本地化）：Employee $\to \cup(E_0, E_{11}, E_{12}, E_{21}, E_{22}, E_{31}, E_{32})$；Loaninfo $\to \cup(L_{11}, \ldots, L_{32})$；Loanbusiness $\to \cup(B_1, B_2, B_3)$。
步骤 2（准则 1：选择作用于片段，矛盾的置空）：

片段	片段条件	与查询条件合取	结果
E_0，E_{2*}，E_{3*}	Bid=A0 / A2 / A3	与 Bid=A1 矛盾	空，删去
E_{12}	Bid=A1 ∧ Etitle≠LO	与 Etitle=LO 矛盾	空，删去
E_{11}	Bid=A1 ∧ Etitle=LO	相容	保留
L_{11}	Bid=A1 ∧ Lamount≥1000	与 Lamount≤500 矛盾	空，删去
L_{2*}，L_{3*}	Bid=A2 / A3	与 Bid=A1 矛盾	空，删去
L_{12}	Bid=A1 ∧ Lamount<1000	相容	保留

步骤 3（准则 3、准则 2）：把 $\bowtie_{\text{Lid}}$ 移到 $\cup(B_1, B_2, B_3)$ 下面，得到 $L_{12} \bowtie B_1$、$L_{12} \bowtie B_2$、$L_{12} \bowtie B_3$。B_2, B_3 中的企业不与 A1 借贷，连接条件不可能满足，所以 $L_{12} \bowtie B_2 = L_{12} \bowtie B_3 = \varnothing$，只保留 B_1。
步骤 4：去掉片段条件已经蕴含的谓词，得到最终的片段查询树。E_{11}（选 A1 上的副本）、L_{12}、B_1 都在 A_1，查询在 A_1 本地完成。



问 不做“每个企业只在一个分行贷款”的假设，只留 B_1 还对吗?

答 也对。L_{12} 里的贷款都是 A1 发放的，借款企业都与 A1 有借贷关系，一定在 B_1 里。$L_{12} \bowtie B_2$ 即使不为空，查出的也只是 B_1 里已有企业的重复元组，所以结果不变。

1.4 2020 年第 1 题（20 分）

原题

某机构由总部 (A_0) 和两个下属分支机构 (A_1 和 A_2) 组成。总部和分支机构都从事科研业务。现对科研项目信息进行数据管理，涉及到的全局关系模式如下：

Branchers(Bid, Bname, Addrs, Mgrno)：机构基本信息表。Bid 为机构编号，Bname 为机构名称，Adrs 是机构地址，Mgrno 是机构负责人员工编号。

Employee(Eid, Ename, Etitle, Bid)：员工基本信息表。Eid 为员工编号，Ename 为员工姓名，Etitle 为员工职务，Bid 是员工所属的机构编号。

Projects(Pid, Pname, Budget, DESC, Security, Pleader)：项目基本信息表。Pid 为项目编号，Budget 为项目金额，DESC 为项目描述信息，Security 为项目是否涉密，Pleader 为项目负责人员工编号。

数据管理的基本需求是：(1) 非涉密项目保存在项目负责人所在的分支机构场地，其中金额超过 100 万元（含）的项目信息在总行保存副本；(2) 涉密项目信息只在总部场地保存；(3) 总部和各分支机构保存各自的员工信息；(4) 涉密项目的负责人信息在总行保存副本；(5) 机构信息仅在总部场地保存。

请回答：1) 按要求对表进行分片。写出分片定义，画出分片树，并指出分片类型；2) 按需求完成分配设计，指出分配类型；3) 查询机构 A_2 员工张龙负责的项目金额不超过 80 万的非涉密项目信息。要求查询结果包括张龙的姓名、项目名称、项目描述信息和项目金额；4) 对 3) 题的查询进行全局优化，要求写出中间过程；5) 在全局优化的基础上进行片段优化，要求写出中间过程。

问 这题和 2019 年那题有什么不同?

答 这题有两处导出。一是项目存在“负责人所在场地”，Projects 里只有 Pleader，场地要到 Employee.Bid 里查，所以 Projects 由 Employee 导出。二是“涉密项目的负责人”要在总部存副本，一个员工是不是涉密项目负责人要到 Projects 里查，所以 Employee 也要由 Projects 导出，再切一次。

问 两个关系互相导出，会不会循环定义?

答 不会，只要都用全局关系来定义。例如 $E_{i1} = \sigma_{\text{Bid='Ai'}}(\text{Employee}) \ltimes_{\text{Eid=Pleader}} \sigma_{\text{Security='涉密'}}(\text{Projects})$，右边都是全局关系，不依赖其他片段。

问 总部员工负责的非涉密项目放在哪里?

答 题目说的是“分支机构场地”，所以默认非涉密项目的负责人都在 A1、A2，答卷里写一句假设。如果总部员工也可能负责非涉密项目，就再加 P_{01}, P_{02} 放在 A0。

参考答卷：1) 分片定义、分片树、分片类型

Branchers：不分片，存放在 A_0。

Projects：水平分片（Security、Budget 上的基本水平分片，加上按负责人 Bid 的导出水平分片）。

$P_S = \sigma_{\text{Security='涉密'}}(\text{Projects})$； 对 $i = 1, 2$:

$P_{i1} = \sigma_{\text{Security='非涉密'} \wedge \text{Budget} \geq 100}(\text{Projects} \ltimes_{\text{Pleader=Eid}} \sigma_{\text{Bid='Ai'}}(\text{Employee}))$,

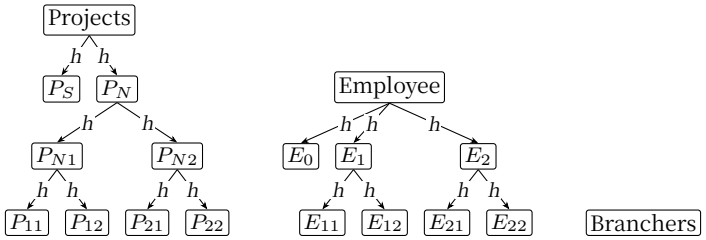
$P_{i2} = \sigma_{\text{Security='非涉密'} \wedge \text{Budget} < 100}(\text{Projects} \ltimes_{\text{Pleader=Eid}} \sigma_{\text{Bid='Ai'}}(\text{Employee}))$。

Employee：水平分片（Bid 上的基本水平分片，加上由 Projects 导出的分片）。$E_0 = \sigma_{\text{Bid='A0'}}(\text{Employee})$; 对 $i = 1, 2$:

$E_{i1} = \sigma_{\text{Bid='Ai'}}(\text{Employee}) \ltimes_{\text{Eid=Pleader}} \sigma_{\text{Security='涉密'}}(\text{Projects})$（A$i$ 中的涉密项目负责人），

$E_{i2} = \sigma_{\text{Bid='Ai'}}(\text{Employee}) - E_{i1}$（A$i$ 的其他员工）。

假设非涉密项目的负责人都属于 A1 或 A2；Bid 只取 A0–A2。



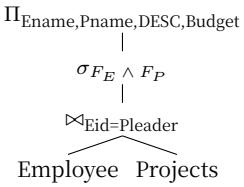
P_N 是非涉密项目，P_{Ni} 是负责人在 Ai 的非涉密项目。

参考答卷：2) 分配设计

A_0（总部）： Branchers，E_0，P_S，E_{11}, E_{21}（副本），P_{11}, P_{21}（副本）
A_1： $E_{11}, E_{12}, P_{11}, P_{12}$
A_2： $E_{21}, E_{22}, P_{21}, P_{22}$
部分片段有两个副本，其余只有一个，是部分复制分配。

参考答卷：3) SQL、关系代数、全局查询树

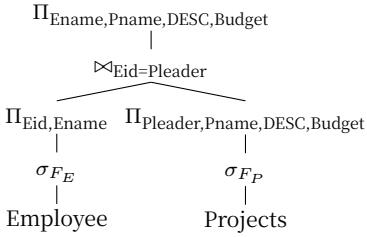
```
SELECT E.Ename, P.Pname, P.DESC, P.Budget
FROM  Employee E, Projects P
WHERE E.Eid=P.Pleader AND E.Ename='张龙'
AND E.Bid='A2' AND P.Budget<=80
AND P.Security='非涉密';
```

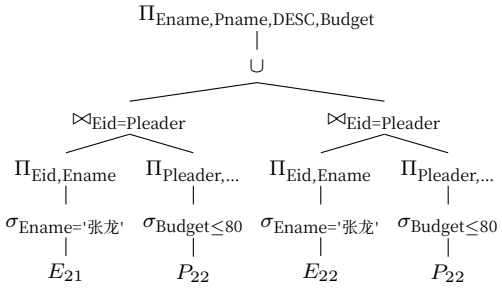


关系代数：$\Pi_{\text{Ename,Pname,DESC,Budget}}\left(\sigma_{F_E \wedge F_P}(\text{Employee} \bowtie_{\text{Eid=Pleader}} \text{Projects})\right)$，
F_E：Ename='张龙'∧Bid='A2'； F_P：Budget≤80∧Security='非涉密'。

参考答卷：4) 全局优化 5) 分片优化

4) 选择串接律拆成 $\sigma_{F_E}\sigma_{F_P}$；选择与连接的分配律把 F_E 移到 Employee 上方，F_P 移到 Projects 上方；用投影的分配律后，Employee 留 {Eid, Ename}，Projects 留 {Pleader, Pname, DESC, Budget}（左树）。
5) 本地化：Employee $\to \cup(E_0, E_{11}, E_{12}, E_{21}, E_{22})$，Projects $\to \cup(P_S, P_{11}, P_{12}, P_{21}, P_{22})$。
准则 1：Bid='A2' 与 E_0, E_{11}, E_{12} 矛盾，删去，留 E_{21}, E_{22}（不知道张龙是否为涉密项目负责人，两个片段都留）； Security='非涉密' 与 P_S 矛盾，Budget≤80 与 P_{11}, P_{21}（Budget≥100）矛盾，删去，留 P_{12}, P_{22}。
准则 3、准则 2：连接移到并下面，得到 4 个连接。P_{12} 的负责人属于 A1，与 A2 的员工按 Eid=Pleader 连接必为空，所以 $E_{2j} \bowtie P_{12} = \varnothing$，只剩 $E_{21} \bowtie P_{22}$、$E_{22} \bowtie P_{22}$。这些片段都在 A_2，本地执行（右树）。





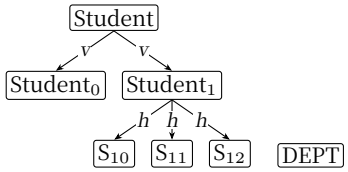
1.5 2023 年第一题

原题

下面是某高校数据库的全局模式：学生 Student = {Sno, Sname, HomeAddr, Phone, Class, Birth, Dno}；学院 DEPT = {Dno, Dname}。该学校共有 3 个学院，Dno 分别为 0，1，2。要求 DEPT 关系和 Student 关系的部分属性 {Sno, Sname, HomeAddr, Phone} 保存在学校场地 S0 上，Student 关系的那部分属性 {Sno, Class, Birth, Dno} 保存在学生所在学院场地 S1、S2 和 S3 上。要求：
1. 将全局模式进行分片，写出分片定义和分片条件；
2. 指出各分片的类型（垂直、水平分片、混合），并画出分片树；
3. 对查询 `SELECT Sname, Class, Dname FROM Student, DEPT WHERE Dno = 2 AND Student.Dno = DEPT.Dno;` 进行全局优化，画出优化后的全局查询树，要求写出中间过程；
4. 进行分片优化，画出优化后的分片查询树，要求写出中间过程。

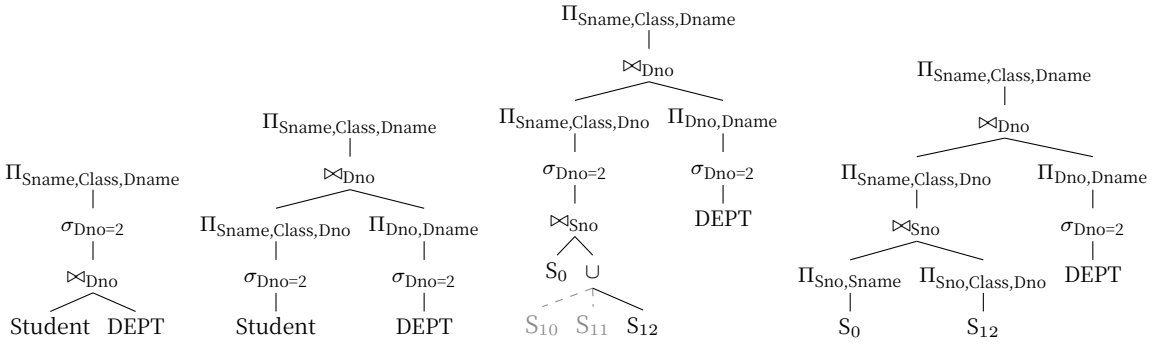
参考答卷：1. 分片定义　2. 类型与分片树

DEPT：不分片，存放在 S0。
Student：先垂直、后水平，是混合分片。
$\text{Student}_0 = \Pi_{\text{Sno,Sname,HomeAddr,Phone}}(\text{Student})$（存 S0）
$\text{Student}_1 = \Pi_{\text{Sno,Class,Birth,Dno}}(\text{Student})$
$\text{Student}_{1j} = \sigma_{\text{Dno}=j}(\text{Student}_1)$，$j = 0, 1, 2$（依次存 S1、S2、S3）
验证：$\text{Attr}(S_0) \cup \text{Attr}(S_1) = \text{Attr}(\text{Student})$，$\text{Attr}(S_0) \cap \text{Attr}(S_1) =$ {Sno}，即主键；三个水平条件互斥且覆盖。
分配：S0 存 DEPT、Student_0；S1 存 Student_{10}；S2 存 Student_{11}；S3 存 Student_{12}。没有片段被复制，是全分割。



参考答卷：3. 全局优化　4. 分片优化

初始查询：$\Pi_{\text{Sname,Class,Dname}}(\sigma_{\text{Dno}=2}(\text{Student} \bowtie_{\text{Dno}} \text{DEPT}))$（图 a）。
全局优化：Dno=2 通过连接条件传递到 DEPT，两边都下移；Student 上方投影 {Sname, Class, Dno}，DEPT 上方投影 {Dno, Dname}（图 b）。
分片优化：① 本地化，$\text{Student} \to \text{Student}_0 \bowtie_{\text{Sno}} \cup(\text{Student}_{10}, \text{Student}_{11}, \text{Student}_{12})$（图 c）；② Dno 只在垂直片段 Student_1 上，$\sigma_{\text{Dno}=2}$ 移进 $\cup$ 后与 Student_{10}、Student_{11} 矛盾，删去，只留 Student_{12}，而且 $\sigma_{\text{Dno}=2}(\text{Student}_{12}) = \text{Student}_{12}$；③ 投影分到两个垂直片段：$\text{Student}_0$ 留 {Sno, Sname}，Student_{12} 留 {Sno, Class, Dno}，保留 Sno 用于垂直重构的连接；HomeAddr、Phone、Birth 用不到。两个垂直片段都提供输出属性，都不能删（图 d）。



(a) 初始　(b) 全局优化后　(c) 本地化，灰色已删去　(d) 分片优化后

第2部分 代价计算：I/O 代价、执行场地、半连接

2.1 I/O 代价（2023 年第三题，课件 5-11–18 的原例）

原题

假设有学生关系 Students{Sno, Sname, Class}，选课关系 SC{Sno, Cno}，选修 5 号课程的学生姓名 SQL 语句如下：

```
SELECT Sname FROM Student, SC WHERE Student.Sno = SC.Sno AND Cno = '5';
```

可以转换成三种关系代数表达式：

$Q_1 = \Pi_{\text{Sname}}(\sigma_{\text{Student.Sno=SC.Sno} \wedge \text{Cno=5}}(\text{Student} \times \text{SC}))$

$Q_2 = \Pi_{\text{Sname}}(\sigma_{\text{Cno=5}}(\text{Student} \bowtie_{\text{Sno}} \text{SC}))$，假设自然连接结果大小为 10^4 个元组

$Q_3 = \Pi_{\text{Sname}}(\text{Student} \bowtie_{\text{Sno}} \sigma_{\text{Cno=5}}(\text{SC}))$

假设内存中存放 5 块 Students 元组和一块 SC 元组，一块可以装 10 个 Students 元组或 100 个 SC 元组。Students 有 1000 个元组，SC 有 10000 个元组，其中选 5 号课程的有 50 个元组，采用循环全连接算法实现，假设 I/O 读写速度是每秒 25 块。计算 Q_1、Q_2 和 Q_3 的 I/O 读写时间代价。

计算步骤

1. 块数 = 元组数 ÷ 每块元组数：Students $1000/10 = 100$ 块，SC $10000/100 = 100$ 块。
2. 嵌套循环连接读的块数 = 外层块数 + 外层批数 × 内层块数，外层批数 = 外层块数 ÷ 外层可用的内存块数。
3. 中间结果放不下内存，就要先写到外存，下一步再读回来，代价是 2 × 中间结果块数。结果能放进内存（如 50 个元组）就不计 I/O。
4. 时间 = 块数 ÷ 每秒块数，各步相加。

问 “SC 被遍历的次数 $= 1000/(10 \times 5) = 20$”是怎么来的？

答 内存一次只能放 5 块 Students，也就是 50 个学生。Students 共 100 块，要分 $100/5 = 20$ 批读入。每读入一批，都要把 SC 从头到尾逐块读一遍（每次读进那 1 块 SC 缓冲区），和这 50 个学生配对。所以 SC 被读 20 遍，每遍 100 块。

问 Q_1 的笛卡尔积为什么要写出去再读回来？

答 $1000 \times 10^4 = 10^7$ 个元组放不进内存，只能先写到磁盘（$10^7/10 = 10^6$ 块），做选择时再读回来，又是 10^6 块，所以算两次。每块装 10 个笛卡尔积元组是课件和参考答案用的假设，答题时要写出来。

问 Q_3 为什么只要 8 秒？

答 它先做选择：读一遍 SC（100 块），筛完只剩 50 个元组，全部放进内存。连接时把 Students 读一遍（100 块），逐个和内存里的 50 个元组配对，不需要反复扫描，也不写中间结果。

参考答卷：2023 年第三题

假设：笛卡尔积和连接的中间结果每块可装 10 个元组。

Q_1：① 读：$\underbrace{1000/10}_{\text{Students 块数}} + \underbrace{1000/(10 \times 5)}_{\text{SC 遍历次数}} \times \underbrace{10000/100}_{\text{SC 块数}} = 100 + 20 \times 100 = 2100$ 块，$2100/25 = 84\,\text{s}$；② 写笛卡尔积：$10^3 \times 10^4 = 10^7$ 个元组，$10^7/10 = 10^6$ 块，$10^6/25 = 4 \times 10^4\,\text{s}$；③ 选择时读回：$4 \times 10^4\,\text{s}$，选择后剩 50 个元组，放在内存；④ 投影不需要 I/O。

共 $84 + 2 \times 4 \times 10^4 = 80084\,\text{s}$，约 22.2 小时。

Q_2：① 读：同上，2100 块，84 s；② 写连接结果：$10^4/10 = 1000$ 块，40 s；③ 选择时读回：40 s；④ 投影不需要 I/O。共 $84 + 40 + 40 = 164\,\text{s}$，约 2.7 分钟。

Q_3：① 选择：读 SC $10000/100 = 100$ 块，4 s，结果 50 个元组在内存；② 连接：读 Students $1000/10 = 100$ 块，4 s，结果 50 个元组在内存；③ 投影不需要 I/O。共 8 s。

结论：Q_3 代价最小。它先选择、再连接，没有笛卡尔积，比 Q_1 快了约一万倍。

注意

课件原例是“每秒读写 20 块、选 2 号课程”，结果分别是 105 s、10^5 s（约 28 小时）、205 s、10 s。考试换了数字，算法相同：先算块数，再除以速度。

2.2 选择执行场地（2023 年第四题，课件 6-5–7 的原例）

原题

有关系 EMP 和 DEPT 如下：EMP{ENO, ENAME, BIRTH, SALARY, DNO}，DEPT{DNO, DNAME}。假设 EMP：元组数 10000 条，元组大小 100 Byte；DEPT：元组数 100 条，元组大小 35 Byte。存在三个场地 S1、S2 和 S3。EMP 存放在 S1 场地，DEPT 存放在 S2 场地，S3 为查询场地。查询每个雇员的姓名及所在单位名称 SQL 语句：`SELECT ENAME, DNAME FROM EMP, DEPT WHERE EMP.DNO = DEPT.DNO;` 关系表达式为 $\Pi_{\text{ENAME,DNAME}}(\text{EMP} \bowtie_{\text{DNO}} \text{DEPT})$。假设结果元组大小 40 字节，S3 为查询场地，结果关系大小：$40 \times 10000 = 400\,\text{KB}$。请选择传输代价最小的查询执行场地。

计算步骤

1. 算每个关系的大小：Size = Card × Length；结果大小 = 结果元组数 × 结果元组长度。
2. 对每个候选执行场地 S_k：传输量 = 不在 S_k 的输入关系大小之和，若 S_k 不是查询场地，再加上结果大小。
3. 取最小的。题目给了 C_0, C_1 时，每次传输的代价是 $C_0 + C_1 \cdot X$。

参考答卷：2023 年第四题

$\text{Size(EMP)} = 10000 \times 100\,\text{B} = 1000\,\text{KB}$；$\text{Size(DEPT)} = 100 \times 35\,\text{B} = 3.5\,\text{KB}$；$\text{Size(R)} = 10000 \times 40\,\text{B} = 400\,\text{KB}$。

在 S1 执行：	传 DEPT 到 S1，结果 R 传到 S3	$3.5 + 400 = 403.5\,\text{KB}$
在 S2 执行：	传 EMP 到 S2，结果 R 传到 S3	$1000 + 400 = 1400\,\text{KB}$
在 S3 执行：	传 EMP、DEPT 到 S3	$1000 + 3.5 = 1003.5\,\text{KB}$

选 S1 为执行场地，传输代价最小，为 403.5 KB。

问 在查询场地 S3 执行看起来最自然，为什么反而不好?

答 EMP 有 1000 KB，整个搬走代价很高。一般把小关系搬到大关系所在的场地，但结果还要送回查询场地，所以结果大小也要算进去再比较。这题结果只有 400 KB，比 EMP 小，所以在 S1 执行更省。

2.3 直接连接与半连接的通信代价（2020、2019 年第 2 题，课件 6-15–39）

估算与比较的步骤

1. 写假设：数据均匀分布、属性相互独立；外键的取值来自被参照关系的主键；只计传输的数据量，或写成 $C_0 + C_1 X$。
2. 先做本地选择，不需要通信。等值条件 $\rho = 1/\text{Val}(R, A)$；范围条件 $\rho = (X - \text{Min})/(\text{Max} - \text{Min})$，也可以按取值个数算；$\text{Card} = \rho \cdot \text{Card}(R)$。
3. 直接连接（全连接）：把一个关系整个传到另一个关系所在的场地做连接，结果再传回查询场地。两个方向都算，取小的。
4. 半连接：① 把 S' 的连接列 $\Pi_{\text{A}}(S')$ 传到 R 的场地，大小为 $\text{Val}(S', A) \times \text{Length}(A)$；② 计算 $R' = R \ltimes S'$，$\rho = \text{Val}(S', A)/\text{Val}(S, A)$，$\text{Card}(R') = \rho \cdot \text{Card}(R)$；③ 把 R' 传回去做连接。
5. 比较：当 $C_0/C_1 + \text{Size}(\Pi_{\text{A}} S') + \text{Size}(R') \le \text{Size}(R)$ 时，半连接比直接传 R 省 [课件6-39]。

问 半连接省在哪里?

答 直接连接要把整个 R 传过去，而 R 里大部分元组连不上。半连接先把对方的连接列（很窄）传过来，筛掉连不上的元组，然后只传能连上的 R'。连不上的元组越多，也就是选择度 ρ 越小，省得越多。

问 什么时候半连接反而不划算?

答 ρ 接近 1（几乎都能连上），或者连接列本身很宽时。多传的 $\Pi_{\text{A}}(S')$ 和多出的一次 C_0 换不来多少缩减。半连接还会增加本地处理代价，在局域网里不一定合算 [课件6-63]。

问 半连接的选择度为什么是 $\text{Val}(S', A)/\text{Val}(S, A)$?

答 R 的外键 A 均匀地取自 S 的 $\text{Val}(S, A)$ 个值。S 筛成 S' 后只剩 $\text{Val}(S', A)$ 个值，R 中外键落在这些值里的比例就是两者之比。课件 SDD-1 例子里的 $\rho_1 = 200/2000 = 0.1$ 就是这样算的 [课件6-51]。

2020 年第 2 题（10 分）原题

科研项目表存储在科技处场地，教师信息表存储在人事处场地，两个场地通过互联网连接。现从人事处场地查询计算机学院教师承担的项目信息。假设数据满足均匀分布，关系概要图如下：

Projects（单位：字节）Card=30000 项，Length=200

	PID	FID	Pname	Sdate
length	15	8	100	8
val	30000	2500	30000	25000
说明	项目编号	负责人编号	项目名称	立项时间

Faculties（单位：字节）Card=3000 人，Length=300

	FID	Fname	Institute
length	8	20	8
val	3000	2800	20
说明	教师编号	教师姓名	学院

计算：(1) 如果采用全连接优化算法，请给出查询执行计划，并计算通信代价。(2) 请给出优化的查询执行策略，并计算通信代价。

参考答卷：2020 年第 2 题

假设：均匀分布；Projects.FID 的取值来自 Faculties.FID；结果是项目信息（Projects 元组，200 B）；只计传输字节数，每次传输另加 C_0。

本地选择：$F' = \sigma_{\text{Institute='计算机'}}(\text{Faculties})$，$\text{Card}(F') = 3000/20 = 150$，$\text{Val}(F', \text{FID}) = 150$。

选择度 $\rho = 150/3000 = 0.05$，计算机学院教师承担的项目数 $= 30000 \times 0.05 = 1500$。

(1) 全连接：

方案 A，在人事处连接：Projects 整个传到人事处，$30000 \times 200 = 6\,000\,000$ B；

方案 B，在科技处连接：F' 传到科技处 $150 \times 300 = 45\,000$ B，连接后结果传回人事处 $1500 \times 200 = 300\,000$ B，共 $345\,000$ B。

采用方案 B，通信代价 $345\,000$ B，即 $2C_0 + 345\,000\,C_1$。

(2) 半连接策略：

① 人事处把 $\Pi_{\text{FID}}(F')$ 传到科技处：$150 \times 8 = 1\,200$ B；

② 科技处计算 $P' = \text{Projects} \ltimes_{\text{FID}} \Pi_{\text{FID}}(F')$，$\text{Card}(P') = 1500$；

③ P' 传回人事处：$1500 \times 200 = 300\,000$ B；④ 人事处计算 $F' \bowtie_{\text{FID}} P'$ 得到结果。

通信代价 $1\,200 + 300\,000 = 301\,200$ B，即 $2C_0 + 301\,200\,C_1$，小于全连接的 $345\,000$ B，是最优策略。

2019 年第 2 题（10 分）原题

校办拟查询 40 岁及以下教师的基本信息和科研信息，以及他们第 6 学期的教学信息。已知：教师的基本信息（Faculties 表）位于人事处、科研项目信息（Projects 表）位于科技处、教学信息（Schedules）位于教务处。各场地通过互联网连接，数据满足均匀分布。关系概要图如下：

Faculties：Card=3000 人，Length=300

	FID	Fname	Fage
length	8	20	1
val	3000	2800	40
说明	教师编号	教师姓名	年龄

Projects：Card=30000 项，Length=200

	PID	FID	Pname	Sdate
length	15	8	100	8
val	30000	2500	30000	25000
说明	项目编号	负责人编号	项目名称	立项时间

Schedules：Card=120000 门次，Length=150

	CID	Cname	Semester	FID
length	15	60	1	8
val	4000	4000	8	2500
说明	课程编号	课程名称	开课学期	主讲教师

学校教师的年龄均匀分布在 [21, 60] 区间。

给出优化的查询策略，并计算查询代价。

参考答卷：2019 年第 2 题

假设：均匀分布、属性相互独立；Projects.FID 与 Schedules.FID 取自 Faculties.FID；年龄取整数 21–60；只计传输字节；校办本地没有数据。

① 本地选择（不需要通信）：

人事处 $F' = \sigma_{\text{Fage}\le 40}(\text{Faculties})$：21–40 共 20 个值，$\rho = 20/40 = 1/2$（按公式 $(40-21)/(60-21) \approx 0.49$，近似 $1/2$），$\text{Card} = 1500$，$\text{Size} = 1500 \times 300 = 450\,000\,\text{B}$；

教务处 $S_6 = \sigma_{\text{Semester=6}}(\text{Schedules})$：$\rho = 1/8$，$\text{Card} = 15\,000$。

② 半连接：$\Pi_{\text{FID}}(F')$（$1500 \times 8 = 12\,000\,\text{B}$）分别传到科技处和教务处，共 $24\,000\,\text{B}$。

科技处 $P' = \text{Projects} \ltimes \Pi_{\text{FID}}(F')$：$\rho = 1500/3000 = 1/2$，$\text{Card} = 15\,000$，$\text{Size} = 3\,000\,000\,\text{B}$；

教务处 $S' = S_6 \ltimes \Pi_{\text{FID}}(F')$：$\text{Card} = 15\,000 \times 1/2 = 7\,500$，$\text{Size} = 7500 \times 150 = 1\,125\,000\,\text{B}$。

③ 把 F', P', S' 传到校办：$450\,000 + 3\,000\,000 + 1\,125\,000 = 4\,575\,000\,\text{B}$，在校办计算 $F' \bowtie_{\text{FID}} P' \bowtie_{\text{FID}} S'$。

总通信代价：$24\,000 + 4\,575\,000 = 4\,599\,000\,\text{B} \approx 4.6\,\text{MB}$，共 5 次传输，即 $5C_0 + 4\,599\,000\,C_1$。

对比：只做本地选择、不用半连接，要传 $450\,000 + 6\,000\,000 + 15000 \times 150 = 8\,700\,000\,\text{B}$；什么都不做，要传 $900\,000 + 6\,000\,000 + 18\,000\,000 = 24\,900\,000\,\text{B}$。两个半连接的收益分别是 $0.5 \times 6\,000\,000\,\text{B}$ 和 $0.5 \times 2\,250\,000\,\text{B}$，都远大于各自 $12\,000\,\text{B}$ 的代价，所以采用。

问 为什么不先在某个场地把三个关系连好，再把结果传给校办？

答 先估结果大小。每个老师平均有 $30000/2500 = 12$ 个项目、$15000/2500 = 6$ 门第 6 学期的课，三表按 FID 连接得到的是项目和课程的组合，约 8–9 万行，每行 600 多字节，约 50 MB，比三个缩减后的关系加起来的 4.6 MB 大得多。结果比输入大时，应该把缩减后的输入送到查询场地再连接。

问 如果题目的意思是两张结果表（教师和科研、教师和教学），策略要变吗？

答 不用变。还是把 F', P', S' 送到校办，分别做两次连接，通信代价一样。

第3部分 分布式事务：子事务、进程模型、2PC 与故障处理

3.1 分解子事务与进程代理模型 [课件7-19–26]

分解子事务、写进程代理模型的步骤

1. 给每一步操作标上它访问的数据所在的场地。一个场地对应一个子事务，写清每个子事务做什么。
2. 先像集中式那样写出整个事务（begin_transaction ... Commit ... end），再拆分，阅卷时容易看清。
3. 拆成根代理和子代理。根代理在源场地，负责 input、条件判断、`Create AGENTi`、`Send(AGENTi,` 参数`)`，最后执行 `Commit`，由它发起两阶段提交。子代理负责 `Receive(ROOT,` 参数`)`，执行本场地的 SQL，检查不通过就 `Abort`。
4. 题目要求画图时，画进程模型：协调者进程（根代理，含全局事务）和各代理者进程（子代理，各含一个子事务）。

问 根代理和子代理是什么关系?

答 每个应用只有一个根代理，它负责全局事务的开始、提交和废弃。只有根代理能创建子代理。所有子事务都能成功提交时，根代理才决定全局提交，否则全局废弃 [课件7-22]。在 2PC 里，根代理是协调者，子代理是参与者。

问 进程模型和服务器模型怎么区分?

答 进程模型为每个子事务在它的场地上建一个代理者进程，另有一个协调者进程。服务器模型在每个场地只建一个服务器进程，轮流为多个全局事务的子事务服务，不同全局事务在同一场地上共用它 [课件7-26]。

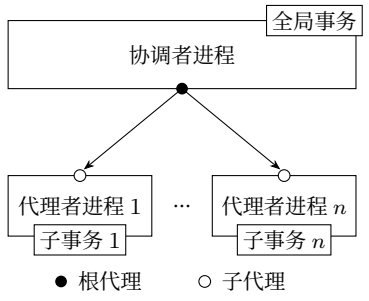
问 子代理里的 Abort 能决定全局废弃吗?

答 不能。它只让本场地的子事务失败。在 2PC 的决定阶段，这个子代理会回答“准备废弃”，全局废弃由协调者决定。

2004 年第 3 题

画出分布式事务进程代理模型，并举实例说明执行过程。

参考答卷：2004 年第 3 题



进程模型：全局事务为每个子事务在相应场地上创建一个代理者进程（子代理），由它执行该场地上的操作；同时启动一个协调者进程（根代理），负责代理者进程之间的通信，控制和协调它们的操作。根代理所在的场地是事务的源场地。只有根代理能创建子代理；所有子事务都成功后，根代理才决定全局提交。

实例：银行转账，ACCOUNT(Acc_No, Amount) 中贷方账户 from_acc 在场地1，借方账户 to_acc 在场地2，根代理在场地1 [课件7-23–25, 7-43–44]。

① 场地1 的根代理向本地 DTM 发 begin_transaction，全局事务开始；② DTM 向本地 LTM 发 local_begin，LTM 写日志后执行局部操作：查 from_acc 的余额，不足则 Abort，否则扣减 transfer_amount；③ 根代理用 create 原语创建子代理 agent1；④ 场地1 的 DTM 通知 agent1 所在场地2 的 DTM，后者向本地 LTM 发 local_begin；⑤ 场地2 的 LTM 写日志；⑥ LTM 激活 agent1；⑦ agent1 用 receive 接收参数 (to_acc, transfer_amount)，执行 `UPDATE ACCOUNT SET Amount=Amount+transfer_amount WHERE Acc_No=to_acc`；⑧ 根代理执行 Commit，用两阶段提交协议提交两个场地上的子事务。

2018 年第三题

存在如下信息：场地1：货品信息表 Article(Ano, Aname, Price, Sold, Total)，用户信息表 User(Uid, name, Tel, Type)；场地2：购买信息表 Purchase(Pid, Uid, Ano, Money)；场地3：电商账户 B_account；场地4：用户账号 U_account。假设用户 A 在某电商的一次购物为一个分布式事务，具体如下：步骤1：客户 A 购买了商品 X，修改该货品 X 的售出数量 Sold；步骤2：写用户 A 的购买信息 Y 到购买信息表 Purchase；步骤3：将用户 A 购买商品 X 的花费从用户账号 U_account 转账到电商账户 B_account。请回答：1. 将该分布式事务分解为相应的子事务；2. 应用进程代理模型概述该业务流程的实现过程；3. 如果采用集中式两段提交协议，写出提交过程。

参考答卷：2018 年第三题

设 U_account(Uid, Balance)、B_account(Bid, Balance)，Total 为库存总数，可售数量为 Total−Sold。

1. 子事务：T_1（场地1）：查 X 的单价和库存，修改 X 的 Sold；T_2（场地2）：向 Purchase 插入 A 的购买记录；T_3（场地3）：电商账户 B_account 加上花费；T_4（场地4）：检查 A 的余额，从 U_account 扣除花费。

2. 根代理在场地1（商品信息所在场地），三个子代理分别在场地4、2、3：

根代理（场地1）

```
begin_transaction
 input(uid, ano, num, bid);
 SELECT Price,Sold,Total INTO :p,:s,:t
   FROM Article WHERE Ano=:ano;
 IF t-s < num THEN
   Output("库存不足"); Abort;
 ELSE
   money := p*num;
   UPDATE Article SET Sold=Sold+:num
     WHERE Ano=:ano;
   Create AGENT1; Send(AGENT1, uid, money);
   Create AGENT2; Send(AGENT2, uid, ano, money);
   Create AGENT3; Send(AGENT3, bid, money);
   Commit;
end
```

AGENT1（场地4）

```
Receive(ROOT, uid, money);
SELECT Balance INTO :b
  FROM U_account WHERE Uid=:uid;
IF b < money THEN Abort;
UPDATE U_account
  SET Balance=Balance-:money
  WHERE Uid=:uid;
```

AGENT2（场地2）

```
Receive(ROOT, uid, ano, money);
INSERT INTO Purchase(Uid,Ano,Money)
  VALUES(:uid, :ano, :money);
```

AGENT3（场地3）

```
Receive(ROOT, bid, money);
UPDATE B_account
  SET Balance=Balance+:money
  WHERE Bid=:bid;
```

3. 协调者是场地1 的根代理，参与者是场地4、2、3 的子代理（场地1 本地的子事务由协调者一并准备和提交）。决定阶段：协调者写“开始提交”记录，向三个参与者发“预提交”；参与者检查本地子事务，能提交就写“准备提交”记录并回答“准备提交”，否则写“准备废弃”并回答“准备废弃”（如 AGENT1 余额不足）。执行阶段：全部回答“准备提交”时，协调者写“决定提交”，发“全局提交”，否则写“决定废弃”，发“全局废弃”；参与者写“提交”或“废弃”记录，提交或撤销子事务，回 Ack；协调者收齐 Ack 后写“事务结束”。共 $4 \times 3 = 12$ 条消息，流程图见 §3.2。

2008 年第 3 题

完成电话银行代缴费业务。设银行账户关系为 ACCOUNT(Acc_no, Amount1)，Acc_no 表示银行存款账号，Amount1 表示银行存款余额；通信公司数据库中的客户关系为 CUSTOMER(Phone_no, Amount2)，Phone_no 表示客户电话号码，Amount2 表示话费余额。具体的业务过程是：电话用户到达银行，填写拟缴费的电话号码和缴费金额，银行先创建从电话用户账号（设为 111）到通信公司存款账号（设为 222）的转账业务，同时将缴费信息发送给通信公司。通信公司接收到缴费信息后，为该电话号码修改话费余额。假设存放账号 111 的信息表、存放账号 222 信息的表以及通信公司客户关系表均不在一个场地上，请按照进程代理模型给出分布式事务的执行过程。

参考答卷：2008 年第 3 题

根代理在存放账号 111 的场地（用户所在银行），AGENT1 在存放账号 222 的场地，AGENT2 在通信公司场地。

根代理（账号 111 所在场地）

```
begin_transaction
 input(from_no, to_no, phone_no, money);
 SELECT Amount1 INTO :temp FROM ACCOUNT
   WHERE Acc_no=:from_no;
 IF temp < money THEN
   Output("余额不足"); Abort;
 ELSE
   UPDATE ACCOUNT SET Amount1=Amount1-:money
     WHERE Acc_no=:from_no;
   Create AGENT1; Send(AGENT1, to_no, money);
   Create AGENT2; Send(AGENT2, phone_no, money);
   Commit;
   Output("缴费完成");
end
```

AGENT1（账号 222 所在场地）

```
Receive(ROOT, to_no, money);
UPDATE ACCOUNT
  SET Amount1=Amount1+:money
  WHERE Acc_no=:to_no;
```

AGENT2（通信公司场地）

```
Receive(ROOT, phone_no, money);
UPDATE CUSTOMER
  SET Amount2=Amount2+:money
  WHERE Phone_no=:phone_no;
```

根代理执行 Commit 时，以两阶段提交协议提交三个场地的子事务：扣款、转入和修改话费余额要么都生效，要么都不生效。

2011 年第 5 题

完成网上购买飞机票业务，过程是：买票人登录航空公司网站，查询指定日期、指定航班的余票信息。如果有剩余机票，则买票人输入身份信息和机票信息从而生成订单，并进行网上支付。支付成功后，买票过程完毕。现假设航空公司数据、航空公司银行账户数据、买票人银行账户数据分布在三个场地上。银行账户关系为 ACCOUNT(Acc_no, Amount)，Acc_no 表示账号，Amount 表示银行存款余额；航空公司数据库中客户关系为 CUSTOMER(ID, Name, Fno)，ID 为乘客的身份证号码，Name 为乘客姓名，Fno 为航班号；FLIGHT(Fno, Fdate, Price, STSold, Capacity)，Fno 为航班号，Fdate 为起飞日期，Price 为航班价格，STSold 为已卖座位数，Capacity 为总座位数。请按照进程代理模型给出分布式事务的执行过程。

参考答卷：2011 年第 5 题

根代理在航空公司场地，AGENT1 在买票人银行场地，AGENT2 在航空公司银行场地。

根代理（航空公司场地）

```
begin_transaction
 input(fdate, fno);
 SELECT Capacity-STSold, Price INTO :left,:price
   FROM FLIGHT WHERE Fno=:fno AND Fdate=:fdate;
 IF left <= 0 THEN
   Output("无余票"); Abort;
 ELSE
   input(id, name, buyer_acc, air_acc);
   Create AGENT1; Send(AGENT1, buyer_acc, price);
   Create AGENT2; Send(AGENT2, air_acc, price);
   UPDATE FLIGHT SET STSold=STSold+1
     WHERE Fno=:fno AND Fdate=:fdate;
   INSERT INTO CUSTOMER VALUES(:id,:name,:fno);
   Commit;
   Output("购票成功");
end
```

AGENT1（买票人银行）

```
Receive(ROOT, buyer_acc, price);
SELECT Amount INTO :b FROM ACCOUNT
  WHERE Acc_no=:buyer_acc;
IF b < price THEN Abort;
UPDATE ACCOUNT
  SET Amount=Amount-:price
  WHERE Acc_no=:buyer_acc;
```

AGENT2（航空公司银行）

```
Receive(ROOT, air_acc, price);
UPDATE ACCOUNT
  SET Amount=Amount+:price
  WHERE Acc_no=:air_acc;
```

余票不足时根代理直接 Abort；买票人余额不足时 AGENT1 废弃本地子事务，在两阶段提交中回答“准备废弃”，全局事务废弃，已改的 STSold 和 CUSTOMER 记录一起撤销。

3.2 集中式两阶段提交协议 2PC [课件7-29–34, 7-47]



参与者状态：初始 →（发出 R/A 后）准备就绪或废弃 →（发出 Ack 后）提交或废弃
集中式 2PC 的消息数为 $4N$，耗时 $4T$；❶–❺ 是故障点，处理方法见 §3.3 的表

2004 年第 2 题

画出集中式 2PC 的通信协议结构，编写集中式的 2PC 提交协议的协调者和参与者的执行算法。

参考答卷：2004 年第 2 题

通信结构见上图：协调者与每个参与者之间一问一答，参与者之间不通信。
协调者。决定阶段：写“开始提交”记录，向所有参与者发“预提交”，等待应答。执行阶段：若有一个“准备废弃”或超时没有收到应答，写“决定废弃”，发“全局废弃”；否则写“决定提交”，发“全局提交”。然后等待所有 Ack，收齐后写“事务结束”。
参与者。决定阶段：收到“预提交”后，判断本地子事务能否提交。能提交就写“准备提交”记录，回答“准备提交”；否则写“准备废弃”记录，回答“准备废弃”。然后等待协调者的决定。执行阶段：收到“全局提交”就写“提交”记录并提交子事务；收到“全局废弃”就写“废弃”记录并撤销子事务。完成后向协调者发 Ack。
两条规则：先写日志，再发消息；参与者回答“准备提交”以后，不能自行废弃，只能等协调者的决定。

2PC 的实现方法 [课件7-47–53]	通信结构	消息数	耗时
集中式	协调者与参与者一问一答，参与者之间不通信	$4N$	$4T$
分布式	始发场地广播预提交，各场地互相广播应答，各自做决定；只有决定阶段	$N(N+1)$	$2T$
分层式	树形结构，逐层转发，协调者是根	$4N$	$4T \sim 2NT$
线性	场地排成链，预提交逐个向后传，提交从链尾逐个传回	$\leq 3(N-1)$	$\leq 2(N-1)T$

3.3 2PC 的故障与处理 [课件8-42–55]

故障点	故障发生的时机	现象	处理
❶	参与者在写 R/A 记录之前	协调者等应答超时	协调者按废弃处理，视同收到 Abort，全局废弃
❷	参与者写了 R/A，还没写 C/A: (a) 应答没发出 (b) 应答已发出	(a) 协调者收不齐应答 (b) 协调者收不齐 Ack	(a) 参与者恢复后写 A 记录，发 A 应答，全局废弃 (b) 协调者重发 C/A 命令，参与者恢复后按命令提交或废弃
❸	参与者写了 C/A: (a) Ack 没发出 (b) Ack 已发出	(a) 协调者收不齐 Ack (b) 正常	(a) 协调者重发 C/A，参与者按日志 redo 或 undo 后回 Ack (b) 不需要处理
❹	协调者写了“开始提交”，还没写 C/A: (a) P 未发出 (b) P 已发出	(a) 无 (b) 参与者已回 R/A，等命令超时	(a) 协调者重启后重新执行 2PC (b) 参与者启动终结协议
❺	协调者写了 C/A，还没写结束记录: (a) 命令未发出 (b) 命令已发出	(a) 参与者等命令超时 (b) 可能漏收 Ack	(a) 参与者启动终结协议 (b) 协调者重启后向所有参与者重发 C/A
通信	丢失 P 报文或 R/A 应答	协调者等应答超时	协调者按废弃处理
通信	丢失 C/A 命令	参与者等命令超时	参与者请求协调者重发，或启动终结协议
通信	丢失 Ack	协调者等 Ack 超时	协调者重发 C/A 命令，要求应答

终结协议 [课件8-52–53]：参与者等不到协调者的命令时使用

选一个参与者 P_T 作新的协调者，向所有参与者询问状态，然后：有的处于“初始”、有的处于“准备就绪”，发 Abort；全部“准备就绪”，发 Commit；至少一个“提交”，发 Commit；至少一个“废弃”，发 Abort。依据是不会出现“初始”和“提交”并存的情况。

问 协调者收不到某个参与者的 Ready 时可以直接废弃，收不到 Ack 时却只能一直重发，为什么?

答 收 Ready 时还在决定阶段，没有人做出最终决定，废弃是安全的。收 Ack 时已经在执行阶段，决定已写入日志并发给了其他参与者，有的参与者可能已经提交，不能再改，只能重发命令，直到对方恢复并确认。

问 参与者写了 Ready 以后宕机，恢复后能自己决定提交或废弃吗?

答 如果应答还没发出，可以写 Abort（见❷(a)）。应答已经发出时就不能自己决定：它处于不确定状态，必须问协调者，或者问其他参与者（终结协议）。这就是 2PC 的阻塞问题，3PC 在中间加了一个“准备提交”阶段来减少阻塞 [**课件7-55–64**]。

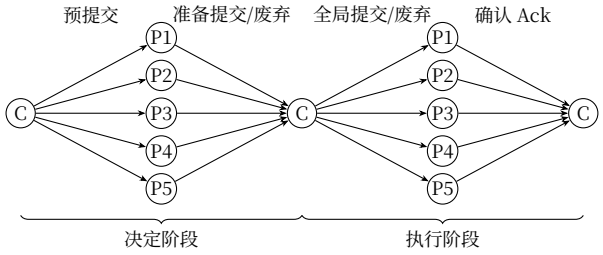
2019 年第 3 题（16 分）

假设存在如下信息。场地1：电子商品信息表 Article(Ano, Aname, Price, Sold, Total)；场地2：电子附件商品信息表 Article(Ano, Aname, Price, Sold, Total)；场地3：购买信息表 Purchaser(Pid, Uid, Ano, Num, Money)，用户信息表 User(Uid, name, Tel, Type)；场地4：电商账户（B_account）；场地5：用户账号（U_account）。假设用户 A 在某电商的一次购物简化为如下一个分布式事务，具体过程：步骤1：客户 A 购买了商品 X（场地1）和商品 Y（场地2），修改商品 X 和 Y 的售出数量 (Sold)；步骤2：写用户 A 的购买信息 XY 到购买信息表 (Purchaser)；步骤3：将用户 A 购买商品 X 和 Y 的花费 Z 从用户账号 (U_account) 转账到电商账户 (B_account) 中。请回答：1) 将该分布式事务分解为相应的子事务；2) 如果采用集中式两阶段提交协议，写出该分布式事务的提交过程；3) 在提交过程中，如果转账没有成功，其提交过程如何？请画图说明故障点，并给出处理策略；4) 如果在发 commit 命令后场地3 出故障，其提交过程又如何？请画图说明故障点，并给出处理策略。

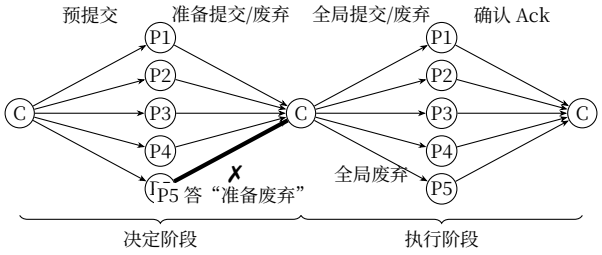
参考答卷：1) 分解子事务　2) 集中式 2PC 的提交过程

设事务由电商平台的源场地 S_0 发起，根代理（协调者）在 S_0。全局事务分解为 5 个子事务：T_1（场地1）：检查 X 的库存，修改 X 的 Sold；T_2（场地2）：检查 Y 的库存，修改 Y 的 Sold；T_3（场地3）：向 Purchaser 插入 A 购买 X、Y 的记录；T_4（场地4）：B_account 余额加 Z；T_5（场地5）：检查 A 的余额，U_account 余额减 Z。

提交过程：协调者 C 是 S_0 的根代理，参与者 P1–P5 是场地1–5 的子代理（下方左图）。决定阶段：C 写“开始提交”，向 P1–P5 发“预提交”。各 P_i 检查本地子事务（P1、P2 库存够，P5 余额够，P3、P4 写入成功），能提交就写“准备提交”并回答“准备提交”，否则写“准备废弃”并回答“准备废弃”。执行阶段：C 收到 5 个“准备提交”，写“决定提交”，发“全局提交”；各 P_i 写“提交”记录，提交子事务，回 Ack；C 收齐 5 个 Ack，写“事务结束”。共 $4 \times 5 = 20$ 条消息。



左：正常提交



右：3) 转账失败，P5 余额不足

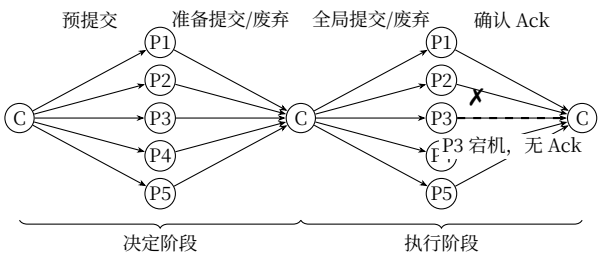
参考答卷：3) 转账没有成功

故障点：决定阶段，P5（U_account）发现余额小于 Z（或者 P4、P5 执行转账失败），本地子事务废弃，见上方右图。

提交过程：P5 写“准备废弃”记录，回答“准备废弃”（P5 宕机没有应答也一样，C 超时后按废弃处理）。C 收到“准备废弃”，写“决定废弃”，向 P1–P5 发“全局废弃”。各参与者写“废弃”记录，撤销本地子事务（P1、P2 恢复 Sold，P3 删除购买记录，P4 恢复余额），回 Ack。C 收齐 Ack 后写“事务结束”。全局事务废弃，所有场地回到购物前的状态。

参考答卷：4) 发出 commit 命令后场地3 出故障

故障点：C 已写“决定提交”并向 P1–P5 发出“全局提交”，P3 宕机（右图）。P1、P2、P4、P5 照常提交并回 Ack，C 收不到 P3 的 Ack。
处理方法取决于 P3 宕机时日志写到了哪里：



① P3 还没写“提交”记录，日志里只有“准备提交”：C 超时后反复重发“全局提交”。P3 恢复后发现子事务处于“准备就绪”的不确定状态，不能自己决定，要向 C 询问，或者向其他参与者询问（终结协议，此时已有参与者提交，所以提交）。得到 Commit 后写“提交”记录，按日志 redo 插入的购买记录并提交，回 Ack。
② P3 已写“提交”记录，还没回 Ack：恢复时把它作为已提交事务 redo；收到 C 重发的命令后回 Ack。
③ P3 已回 Ack：不需要处理。
全局事务仍然提交，其他场地不受影响。C 收齐包括 P3 在内的所有 Ack 以后才写“事务结束”。

问 如果场地3 正好是协调者所在场地呢?

答 那是故障点 ❺。协调者写了“决定提交”并发出命令后宕机，恢复后读日志，看到“决定提交”而没有“事务结束”，就向所有参与者重发“全局提交”，收齐 Ack 后写结束记录。如果宕机时命令还没发出，参与者等待超时后启动终结协议。

2020 年第 4 题（9 分）

某公司根据业务需求设置了 4 个部门，分别为总部、商品研发部、商品生产部和商品销售部，分别在沈阳（场地0）、北京（场地1）、上海（场地2）和广州（场地3）四个城市。原题附图：场地0 总部存“雇员”，场地1 商品研发部存“研发型商品”，场地2 商品生产部存“生产型商品”，场地3 商品销售部存“销售明细”。假设存在如下全局关系模式：雇员（雇员号，姓名，性别，部门）；商品（商品号，商品名，品牌，商品类型，商品负责人），商品类型分为研发型和生产型，对应的负责人为研发负责人和生产负责人；销售明细（商品号，销售数量，销售人员）。假设总部发布一个全局事务 T1：

```
Begin transaction T1  Select 商品类型, count(*) from 商品 group by 商品类型  End
```

请回答：1) 执行该全局事务时，请说明：该全局事务分为几个子事务；定义协调者和参与者，并说明它们完成的功能；2) 若采用集中式两阶段提交协议提交，简述提交过程；3) 在提交过程中，如果丢失了 commit 命令，如何处理?

参考答卷：2020 年第 4 题

1) 商品按商品类型水平分片，研发型存场地1，生产型存场地2，所以 T1 分为 2 个子事务：T_{11}（场地1）：`SELECT '研发型', count(*) FROM 研发型商品`；T_{12}（场地2）：`SELECT '生产型', count(*) FROM 生产型商品`。
协调者：总部场地0 的根代理（事务的源场地）。它分解全局事务，创建子代理并发送子查询，收集两个计数，合并成分组结果返回给用户，然后发起 2PC，决定全局提交或废弃。
参与者：场地1、场地2 的子代理。它们执行本地子事务，返回计数；在 2PC 中报告“准备提交”或“准备废弃”，按协调者的命令提交或废弃，并回 Ack。
2) 协调者写“开始提交”，向场地1、2 发“预提交”；两个参与者写“准备提交”并应答；协调者写“决定提交”，发“全局提交”；参与者写“提交”记录、提交、回 Ack；协调者写“事务结束”。共 $4N = 8$ 条消息，耗时 $4T$。（T1 只读，实际系统可以对只读参与者做优化，答题时按标准流程写。）
3) 参与者回答“准备提交”后收不到命令，处于等待状态。超时后，它请求协调者重发命令，或者启动终结协议：询问另一个参与者的状态，对方已提交就提交，双方都是“准备就绪”也提交。协调者收不到该参与者的 Ack，超时后重发 Commit，直到收到 Ack 再写“事务结束”。

第4部分 并发控制：冲突、局部与全局可串行化、2PL

4.1 判定与构造的方法 [课件9-10–18]

可串行化判定与构造的步骤

1. 把每个事务写成 $R_i(x)$、$W_i(x)$ 序列，并标出场地。“修改 x”写成 $R_i(x)W_i(x)$；“插入”是写；“查询、统计”是读。
2. 找冲突对：同一场地、同一数据项、不同事务、至少一个是写，即 R–W、W–R、W–W；R–R 不冲突。
3. 判断局部可串行：在每个场地的局部历程 $H(S_k)$ 里，冲突对 $O_i < O_j$ 给出一条边 $T_i \to T_j$。没有环就可串行，按拓扑序写出 $SR(H)$。
4. 判断全局可串行：每个局部历程都可串行，并且存在一个与所有场地的边都一致的总序；等价于把各场地的边画在一张图上没有环 [课件9-17]。
5. 构造“局部可串行、全局不可串行”：找两个在两个场地上都有冲突的事务 T_a, T_b，让场地 1 上 T_a 在前、场地 2 上 T_b 在前。每个场地内部写成串行的，局部可串行就很明显。

问 为什么由冲突对决定顺序?

答 两个不冲突的相邻操作交换位置，不影响结果；冲突操作交换以后，读到的值或最后写入的值会变。所以只要每对冲突操作的先后与某个串行历程一致，两个历程就等价（定理 1 的引理 [课件9-15]）。

问 每个场地都可串行了，全局为什么还可能不可串行?

答 各场地可能对应不同的顺序。比如场地 1 等价于 T_1T_3，场地 2 等价于 T_3T_1。每个场地单看都没问题，但没有一个全局顺序能同时满足两者，T_3 看到的是做了一半的 T_1。

问 构造反例需要什么条件?

答 合并后的图要有环，至少需要两个事务在两个不同场地上都有冲突。如果冲突关系是星形的，例如 T_1 和 T_2 只在场地 1 冲突、T_1 和 T_3 只在场地 2 冲突，合并后的图不会有环，怎么排都构造不出来。另外要注意时间上能实现：同一事务里先发生的操作，不能因为在另一个场地的排法而变成后发生。

问 数据项按元组算还是按属性算?

答 按封锁粒度定，一般写“以元组（记录）为数据项”。按元组算时，“改 X 的 Sold”和“读 X 的 Total”冲突，因为是同一条记录 X；按属性算则不冲突。题目要求构造全局不可串行的历程时，通常要按元组算。

4.2 2023 年第五题

原题

设数据项 x，y 存放在 S1 场地，u，v 存放在 S2 场地。有分布式事务 T_1 和 T_2，判断下面的每个执行是否是局部可串行的? 并分别说明理由。在 S1 场地执行历程 H_1：$R_1(x)R_1(y)W_1(x)R_2(x)W_1(y)W_2(x)$；在 S2 场地执行历程 H_2：$R_1(u)R_1(v)R_2(u)W_1(u)W_1(v)W_2(u)$。

参考答卷：2023 年第五题

H_1：x 上的冲突对是 $R_1(x) < W_2(x)$、$W_1(x) < R_2(x)$、$W_1(x) < W_2(x)$，都是 $T_1 \to T_2$；y 只被 T_1 访问。没有环，H_1 是可串行的，等价于串行历程 T_1T_2：

$R_1(x)R_1(y)W_1(x)W_1(y)R_2(x)W_2(x)$。

H_2：u 上的冲突对是 $R_1(u) < W_2(u)$（$T_1 \to T_2$）、$R_2(u) < W_1(u)$（$T_2 \to T_1$）、$W_1(u) < W_2(u)$（$T_1 \to T_2$）。出现 $T_1 \to T_2 \to T_1$ 的环，H_2 不可串行：它不等价于 T_1T_2（$R_2(u) < W_1(u)$ 的次序不同），也不等价于 T_2T_1（$R_1(u) < W_2(u)$ 的次序不同）。因此这组分布式事务的执行也不是全局可串行的。

4.3 2019 年第 4 题（8 分）

原题

针对题 3 信息，假设存在如下三个并发事务：事务 T_1 为客户 A 购买了商品 X（场地1）和商品 Y（场地2），并修改商品 X 和 Y 的售出数量 (Sold)；事务 T_2 为仓储管理者 B 入库商品 X（场地1），并修改商品 X 的数量 (Total)；事务 T_3 为电商经理 C 查询商品 X（场地1）和商品 Y（场地2）的库存数量 (Total)。请回答：1) 几个场地上存在冲突操作？若存在冲突操作，请写出冲突对；2) 针对上述三个分布式事务，请构建一全局历程 H，同时写出：a) 局部可串行化、全局也可串行化的局部历程；b) 局部可串行化、全局不可串行化的局部历程。

参考答卷：2019 年第 4 题

读写序列（以商品元组为数据项，“修改”是先读后写）：

T_1：$R_1(X)W_1(X)$（场地1），$R_1(Y)W_1(Y)$（场地2）；　T_2：$R_2(X)W_2(X)$（场地1）；　T_3：$R_3(X)$（场地1），$R_3(Y)$（场地2）。

1) 场地1 和场地2 上都有冲突操作。

场地1：　T_1–T_2：$(R_1(X), W_2(X))$、$(W_1(X), R_2(X))$、$(W_1(X), W_2(X))$；

T_1–T_3：$(W_1(X), R_3(X))$；　T_2–T_3：$(W_2(X), R_3(X))$。

场地2：　T_1–T_3：$(W_1(Y), R_3(Y))$。$R_1(Y)$ 与 $R_3(Y)$ 都是读，不冲突。

2a) 局部可串行、全局也可串行：

$H(S_1) = R_1(X)W_1(X)R_2(X)W_2(X)R_3(X)$，$T_1 < T_2 < T_3$；　$H(S_2) = R_1(Y)W_1(Y)R_3(Y)$，$T_1 < T_3$。

全局历程 $H = R_1(X)W_1(X)R_1(Y)W_1(Y)R_2(X)W_2(X)R_3(X)R_3(Y)$。总序 $T_1 < T_2 < T_3$ 与两个场地一致，全局可串行。

2b) 局部可串行、全局不可串行：

$H(S_1) = R_1(X)W_1(X)R_2(X)W_2(X)R_3(X)$，串行，$T_1 < T_2 < T_3$；　$H(S_2) = R_3(Y)R_1(Y)W_1(Y)$，串行，$T_3 < T_1$。

全局历程 $H = R_3(Y)\,R_1(X)W_1(X)\,R_1(Y)W_1(Y)\,R_2(X)W_2(X)\,R_3(X)$。

两个局部历程都可串行，但场地1 要求 $T_1 < T_3$，场地2 要求 $T_3 < T_1$，不存在一致的总序，所以全局不可串行。C 读到的 X 是 A 购买之后的值，Y 却是购买之前的值，两者不一致。

4.4 2020 年第 5 题（8 分）

原题

接题 4，如果存在另两个全局事务 T_2 和 T_3：

```
Begin transaction T2  Insert into 商品 values("010","R010"," ","研发型","王一")  End
Begin transaction T3  Insert into 商品 values("110","B110"," ","生产型","李二")  End
```

假设 T_1、T_2、T_3 并发执行，请回答：1) 指出存在冲突操作的场地，并写出存在的冲突对；2) 构建一个局部可串行化，而全局不可串行化的调度历程，并说明为什么？

问 按字面写读写序列是什么样？

答 记 A 为场地1 的研发型商品片段，B 为场地2 的生产型商品片段。统计要读整个片段，所以以片段为数据项。于是 T_1：$R_1(A)$、$R_1(B)$；T_2：$W_2(A)$；T_3：$W_3(B)$。

问 这样能构造出“局部可串行、全局不可串行”吗？

答 只看字面不能。T_1 和 T_2 只在场地1 冲突，T_1 和 T_3 只在场地2 冲突，T_2 和 T_3 不冲突。冲突关系是星形的，合并后最多两条边，不会有环。要构造反例，需要补上插入的语义：插入时要检查商品号（主码）全局唯一，所以 T_2 要到场地2 读 B 查重（$R_2(B)$），T_3 要到场地1 读 A 查重（$R_3(A)$）。这样 T_2、T_3 在两个场地都冲突，就可能有环。答卷里要写出这个假设。

参考答卷：2020 年第 5 题

1) 冲突场地与冲突对（A、B 的含义同上）：场地1：$(R_1(A), W_2(A))$；场地2：$(R_1(B), W_3(B))$。场地0、场地3没有冲突。

如果考虑插入时的主码唯一性检查（T_2 读 B，T_3 读 A），还有：场地1 $(R_3(A), W_2(A))$，场地2 $(R_2(B), W_3(B))$。

2) 构造（在主码检查的假设下；按字面，冲突关系是星形的，不会成环，理由见上面的问答）：

$T_1 = \{R_1(A), R_1(B)\}$，$T_2 = \{R_2(B), W_2(A)\}$，$T_3 = \{R_3(A), W_3(B)\}$，T_2、T_3 都是先查重、后插入。

$H(S_1) = R_3(A)\, R_1(A)\, W_2(A)$：冲突 $R_3(A) < W_2(A)$、$R_1(A) < W_2(A)$，等价于串行 $T_3T_1T_2$，可串行；

$H(S_2) = R_2(B)\, R_1(B)\, W_3(B)$：冲突 $R_2(B) < W_3(B)$、$R_1(B) < W_3(B)$，等价于串行 $T_2T_1T_3$，可串行。

全局的时间顺序可以是 $R_2(B)\, R_3(A)\, R_1(A)\, R_1(B)\, W_2(A)\, W_3(B)$，每个事务都先读后写，时间上可以实现。

全局不可串行的原因：场地1 要求 $T_3 < T_2$，场地2 要求 $T_2 < T_3$，两者矛盾，不存在满足所有局部先后关系的总序。两个事务都在对方插入之前查重，如果两条记录的主码相同，就会插入重复的主码。

4.5 两阶段封锁协议（2023 年第六题）[课件9-24–33]

原题

叙述两阶段封锁协议 2PC 的基本思想，分别说明使用 2PC 协议进行集中式和分布式并发控制实现方法。

参考答卷：2023 年第六题

基本思想：把事务的加锁和解锁分成两个阶段。加锁阶段，事务读写数据前必须先加相应的锁（读锁共享，写锁排斥），申请不到相容的锁就等待；解锁阶段，事务一旦释放了某个锁，就不能再申请新的锁。所有事务都遵守两阶段封锁协议时，并发调度是可串行化的。严格的两阶段封锁协议把锁一直保持到 commit 或 abort 之后再释放，可以避免级联废弃。

集中式实现（整个系统只有一个 2PL 调度器，在中心场地）[课件9-31]：① 协调者 TM 向中心场地的锁管理器 LM 发加锁请求；② LM 判断相容性，相容就允许加锁，否则让事务等待；③ TM 把操作发给各数据场地的数据处理器 DP 执行；④ DP 返回操作结束；⑤ TM 通知 LM 释放锁。优点是实现简单、便于管理；缺点是中心场地负载重，存在单点失效。

分布式实现（每个场地有自己的 2PL 调度器）[课件9-32]：① 协调者 TM 把加锁请求发给数据所在场地的 LM；② 各 LM 加锁成功后，直接把操作交给本场地的 DP 执行；③ DP 执行完后通知协调者 TM 操作结束；④ TM 通知各参与场地的 LM 释放锁。优点是没有中心场地的瓶颈和单点失效；缺点是锁管理复杂，可能出现分布式死锁。

有副本时的封锁方法 [课件9-33]：主副本法，锁只加在主副本上；主场地法，锁都加在主场地上；读写全法，读锁封锁一个副本，写锁封锁全部副本；多数副本法，读锁和写锁都要封锁一半以上的副本。

注意

题目写的是“两阶段封锁协议 2PC”，内容是封锁（Locking），一般缩写为 2PL；两阶段提交协议（Commit）才是 2PC。按封锁协议作答，不要写成预提交、全局提交的流程。2023 年参考答案在这里写错了。

第5部分 故障恢复：日志、检查点、undo 与 redo

由日志求 undo 表和 redo 表的步骤

1. 记号：B_i 开始，D_i^j 是 T_i 的第 j 个数据操作，C_i 提交，A_i 废弃，K 是检查点 [课件8-21]。
2. 画并发执行图：每个事务一行，横轴按日志顺序排列，用竖线标出 K 和故障点。
3. 初始化：undo 表 = 检查点时刻的活动事务（在 K 之前开始、在 K 之前没有 C 或 A 的事务），redo 表为空。
4. 从 K 向后扫描到日志末尾：遇到 B_i，把 T_i 加入 undo 表；遇到 C_i 或 A_i，把 T_i 从 undo 表移到 redo 表（课件 8-39 和课件末尾速查表都按这个口径）。
5. 执行：从日志末尾反向扫描，反做 undo 表中的事务；再从 K 正向扫描，重做 redo 表中的事务。K 之前已经结束的事务不用处理。

问 检查点之前就提交了的事务为什么不用管?

答 检查点要做四件事：① 把日志缓冲区写到外存日志；② 在外存日志里登记检查点记录；③ 把数据库缓冲区写到外存数据库；④ 把检查点记录在日志中的地址写入重启动文件 [课件8-18]。第 ③ 步完成后，检查点之前提交的事务的修改都已写入磁盘，所以恢复只需从最近的检查点开始。

问 有 A_i 记录的事务，为什么放进 redo 表而不是 undo 表?

答 A_i 说明故障前这个事务已经做完了回滚，回滚时写的数据也记在日志里，所以它和提交的事务一样已经结束。恢复时按日志把它重做一遍，先做的修改和后做的撤销都重放，数据库就回到故障前的状态。课件 8-39、课件末尾的速查表和 2023 年参考答案都这样处理。课件 8-38 那一页只把它从 undo 表删掉、不放进 redo 表，两页不一致，答题按 8-39 写。

问 undo 和 redo 分别保证什么?

答 undo 把未提交事务已写入外存的数据恢复成前像，保证原子性；redo 把已提交但还没写入外存的数据恢复成后像，保证持久性。两者都是幂等的，做几次结果都一样 [课件8-22–23]。

2023 年第二题

下面是当一个数据库系统出现故障时，日志文件中的信息： $B_1\ B_2\ D_1^1\ D_2^1\ C_1\ K\ B_3\ D_3^1\ C_3\ D_2^2\ B_4\ A_5\ D_4^1$ Crash
根据上述 log 信息，完成下面的处理：1. 画出对应的事务并发执行图；2. 说明检查点的作用和检查点时刻数据库需要完成的主要操作；3. 确定出反做 (undo) 和重做 (redo) 事务集（写出详细过程）；4. 叙述 undo 和 redo 思想，详细写出其基于日志的数据处理过程。

参考答卷：2023 年第二题

说明：日志中的废弃记录印作 A_5，但 T_5 没有开始记录。这道题和课件 8-38、8-39 的例题相同，那里是 A_2。下面按 A_2 作答，第 3 问末尾附按 A_5 理解时的结果。

1. 并发执行图

	1	2	3	4	5		7	8	9	10	11	12	13	
T_1	B_1		D_1^1		C_1									
T_2		B_2		D_2^1		K				D_2^2		A_2		Crash
T_3							B_3	D_3^1	C_3					
T_4											B_4		D_4^1	

2. 检查点是在日志中周期性设置的操作标志，作用是减少故障恢复的工作量，恢复时只需从最近的检查点开始。在检查点上要完成：① 把日志缓冲区的内容写入外存日志；② 在外存日志中登记一个检查点记录；③ 把数据库缓冲区的内容写入外存数据库；④ 把外存日志中检查点的地址写入重启动文件。检查点记录里还登记当时的活动事务清单。

3. ① 从重启动文件找到最后一个检查点 K。K 时刻 T_1 已提交，活动事务只有 T_2，初始化 undo 表 = $\{T_2\}$，redo 表 = {}。② 从 K 正向扫描：

遇到 B_3： T_3 开始，加入 undo 表 undo = $\{T_2, T_3\}$，redo = {}
遇到 C_3： T_3 提交，移入 redo 表 undo = $\{T_2\}$，redo = $\{T_3\}$
遇到 B_4： T_4 开始，加入 undo 表 undo = $\{T_2, T_4\}$，redo = $\{T_3\}$
遇到 A_2： T_2 废弃，移入 redo 表 undo = $\{T_4\}$，redo = $\{T_3, T_2\}$

③ 扫描到日志末尾。结论：反做 T_4，重做 T_3、T_2；T_1 在检查点之前已提交，不处理。

如果确实是 A_5，只能认为 T_5 在这段日志之前就已开始，检查点时处于活动状态：undo 表初值为 $\{T_2, T_5\}$，最后 undo = $\{T_2, T_4\}$，redo = $\{T_3, T_5\}$。

4. undo（反做、回滚）把数据项恢复到修改之前的值（前像），用于未提交、但修改可能已写入外存的事务，保证原子性。redo（重做、前滚）把数据项恢复到修改之后的值（后像），用于已提交、但修改可能还没写入外存的事务，保证持久性。 基于日志的处理过程（no_fix/no_flush 策略）：① 局部恢复管理器 LRM 从重启动文件取得最近检查点的地址，建立重做表和反做表，重做表初值为空，反做表初值为检查点上的活动事务；② 从检查点向日志末尾扫描，遇到新开始的事务放入反做表，遇到提交或废弃的事务从反做表移到重做表；③ 反向扫描日志，反做反做表中的事务，直到这些事务的 B 记录；④ 从检查点正向扫描，重做重做表中的事务，直到相应的 C 记录。

缓冲区更新策略 [课件8-32–37]	特点	恢复时需要
fix / flush	执行中页面固定，未提交的修改不写盘；结束时强制写盘	都不需要
fix / no_flush	未提交的修改不写盘；已提交的修改不一定写盘	只要 redo
no_fix / flush	未提交的修改可能写盘；已提交的修改一定写盘	只要 undo
no_fix / no_flush	两种情况都可能出现	undo 和 redo，即上面的流程

第6部分 数据复制与一致性协议

6.1 主从同步复制（2020 年第 3 题）[课件10-11–28]

2020 年第 3 题（8 分）原题

假设单主站点的情况下，有三个节点 A、B、C，其中 A 保存着 x 和 y 的主副本，B 和 C 保存着它们的副本，考虑下面事务：
T_1：Read(x)　Write(x)　Commit
假设 T_1 在 B 上提交，请回答：1) 结合同步复制协议的执行方式；2) 解释如何保证数据一致性。

问 同步复制和异步复制的区别在哪里?

答 看事务提交前是否等所有副本都更新完。同步复制在同一个事务里把更新传到全部副本，提交时所有副本的值相同，也就是读一写全（ROWA），能保证强一致；缺点是慢，受最慢的节点拖累，某个副本坏了事务就完成不了。异步复制先在主副本上更新并提交，之后再刷新其他副本，响应快、可用性高，但可能读到旧数据，还可能出现“事务倒转” [课件10-11–12]。

问 T_1 在从节点 B 上提交，B 能直接写自己的副本吗?

答 不能。单主节点结构下，封锁和更新都先作用在主副本上。B 上的事务管理器 TM 负责协调：读写请求先交给主节点 A 加锁，由 A 确定冲突更新的先后，再由 TM 把写操作发到所有从副本。这是完全复制透明的协议，应用只需把事务交给本地 TM [课件10-27–28]。

参考答卷：2020 年第 3 题

1) 执行方式（主从结构加同步复制，B 的 TM 作协调者，完全复制透明）：
① Read(x)：B 的 TM 把读请求发给主节点 A；A 在主副本 x_A 上加读锁，通知 TM；TM 把 Read(x) 转给任意一个存有 x 副本的节点执行，一般就在本地 B 读 x_B。
② Write(x)：TM 把写请求发给 A；A 给 x_A 加写锁，在主副本上执行 Write(x)，通知 TM 已获得写锁；TM 再把 Write(x) 发给存有 x 从副本的 B、C，它们在本地副本上执行写操作。冲突的更新在各从节点上的执行顺序与主节点相同。
③ Commit：同步复制要求所有副本一起提交，所以由 B 的 TM 作协调者，A、B、C 作参与者执行 2PC：预提交、准备提交、全局提交、Ack。提交完成后 A 释放 x 上的锁。
如果采用有限复制透明，即应用知道主节点的位置：T_1 的读写直接交给 A 执行，A 加锁、写主副本，再由主节点的 TM 把 Write(x) 发到 B、C，A 担任 2PC 的协调者 [课件10-25]。
2) 保证一致性的方法：① 对 x 的锁都加在主副本上，冲突操作在 A 上排好先后，各从副本按同样的顺序执行；② 写操作在同一个事务内传到 A、B、C 的全部副本（ROWA），读任何一个副本都能读到最新值；③ 用 2PC 原子提交，所有副本要么都提交新值，要么都不提交，不会只有部分副本更新；④ 锁保持到提交之后才释放（严格 2PL），其他事务读不到中间状态。所以提交后各副本的值相同，达到强一致性。

	主从式复制	对等式复制
更新地点	只能在主副本上更新，再传播到从副本	任一副本都可以更新，再传播到其他副本
优点	实现简单，主副本上总是最新值，容易维护一致性	没有单点瓶颈，负载均衡，可用性高
缺点	主节点负载重，是瓶颈；自治性低	同一数据可能在多处同时更新，要做冲突消解，开销大

6.2 Paxos、反熵、NWR 的比较（2018 年第五题）[课件10-38–54]

原题

对比分析维护多副本一致性协议 Paxos、反熵、NWR 的异同，请写出：各自的优势和不足？分别适合于什么场景？

参考答卷：2018 年第五题

相同点：三者都通过副本之间传递消息，让多个副本的数据达到一致，都允许部分节点失效。不同点在于一致性的强弱（强一致、最终一致、可配置）和协调方式（多数派投票、节点两两随机交换、读写法定数）。

	Paxos	反熵（Gossip 的一种）	NWR
思想	通过消息传递从多个冲突的提议中选出一个。两个阶段：Prepare(n) 得到多数 acceptor 的承诺，Accept(n, v) 得到多数批准。角色有 proposer、acceptor、learner	节点周期性地随机选择其他节点，交换 (key, value, version) 摘要，以 push、pull 或 push-pull 方式按版本号更新	共 N 个副本，写操作至少写成功 W 个，读操作至少读 R 个。$W+R>N$ 时读集合和写集合必定相交，至少能读到一个最新版本
一致性	强一致，由多数派决定	最终一致	可配置：$W+R>N$ 时强一致，否则较弱
优势	强一致；能容忍少数节点故障和消息丢失、重复（非拜占庭模型）	去中心化，节点可以随时加入或退出，容错性好，信息按指数速度传播	可以按需求设定读写性能和一致性级别；吞吐量高，容易扩展
不足	算法复杂，难以实现；需要多轮消息，开销大；不能处理恶意节点	有消息延迟和冗余，不能保证实时一致	参数配置复杂；$W+R\le N$ 时可能读到旧值；$W=N$ 时写操作的可用性差
场景	对一致性要求高的系统，如分布式数据库、选主、锁服务（Chubby、Spanner）	实时性要求低、最终一致即可的场景，如成员管理、分布式存储和备份、Dynamo 和 Cassandra 的副本同步	需要在读写性能和一致性之间折中的键值存储，常取 $N=3$、$W=R=2$

问 为什么 $W+R>N$ 就能读到最新值？

答 写成功的 W 个副本和读到的 R 个副本如果没有交集，就要占用 $W+R$ 个不同的副本。$W+R>N$ 时副本不够，所以两者至少有一个公共副本，它上面是最新版本，再按版本号挑出最新的值。

问 Raft 和 Paxos 是什么关系？

答 两者都解决非拜占庭环境下的一致性问题，Raft 更容易理解。Raft 的节点分为 Leader、Follower、Candidate：Follower 收不到 Leader 的消息就变成 Candidate 发起投票，得票过半的成为 Leader，再由 Leader 向 Follower 同步日志 [课件 10-43–44]。

第7部分 概念题

概念题

理解分布式事务 ACID 四个特性，说明它们同集中式数据库的异同。此外，分布式事务的实现模型主要有哪些?

参考答卷

原子性：事务的操作要么全做，要么全不做。分布式事务要求所有子事务要么全部提交，要么全部撤销，由 2PC 保证。 一致性：事务使数据库从一个一致状态变到另一个一致状态。分布式环境下还包括多个副本之间的一致。
隔离性：一个事务的执行不受其他事务干扰，未提交的结果不暴露给其他事务，以避免级联废弃。分布式环境下要求全局可串行，即各场地局部可串行，并且存在一致的总序。 持久性：事务提交后，修改永久保存，发生故障后可以 redo 恢复。分布式环境下还要处理场地故障和通信故障。
异同：分布式事务继承了集中式事务的 ACID 特性。不同在于它由多个场地上的子事务组成，需要协调各子事务的提交与撤销；除了数据的存取操作序列，还涉及大量通信原语和控制报文 [课件7-20]。
实现模型：进程模型为每个子事务在相应场地建立代理者进程，另设协调者进程（根代理）负责通信与协调，只有根代理能创建子代理并决定全局提交或废弃；服务器模型在每个执行场地建立一个服务器进程，轮流为多个全局事务的子事务服务。

2005 年第 5.1 题

浅析分布式数据库与集中式数据库的主要区别。

参考答卷

数据存储：集中式数据库的数据都在一个场地；分布式数据库在逻辑上是一个整体，物理上分散在多个场地，需要做分片、分配和复制设计。 查询处理：集中式主要考虑 I/O 和 CPU 代价；分布式还要考虑网络传输代价，广域网中以传输代价为主，会用到半连接等技术。 事务管理：分布式事务分解为多个子事务，要用 2PC 保证原子性。 并发控制：分布式要保证全局可串行，还要处理分布式死锁和副本的封锁。 恢复：分布式多了场地故障和通信故障，2PC 存在阻塞问题，需要 3PC 或终结协议。 可靠性和扩展性：分布式数据库有副本，可用性和可靠性更高，容易扩展，可以并行处理；代价是系统复杂、维护开销大、安全问题更多 [课件1-53–55]。

2019 年第 5 题（6 分）

在区块链技术中，如何维护多点的数据一致性？如何理解事务特性?

参考答卷

一致性：每个节点保存账本的完整副本，相当于全复制、对等式的分布式系统。新交易广播到全网，节点用共识算法决定由谁打包下一个区块，并让各节点接受同一个区块。公有链用 PoW、PoS 等算法，可以容忍恶意节点（拜占庭将军问题）；联盟链常用 PBFT、Raft。区块之间用哈希指针相连，篡改任何一个历史区块，后面所有区块的哈希都会对不上。出现分叉时按最长链规则收敛，所以是最终一致，交易需要等待若干个确认块才算确定。
事务特性：一笔交易就是一个事务。原子性：交易要么完整地打包进区块，要么不生效。一致性：交易要通过签名、余额等规则的验证，防止双花。隔离性：区块内的交易排成全序依次执行，相当于串行调度。持久性：区块被确认、后面又有足够多的区块之后，几乎无法篡改。与 2PC 相比，区块链不依赖中心协调者，用共识代替原子提交；代价是吞吐量低、确认延迟大，而且只能做到最终一致。

2005 年第 5.2 题、2006 年第 6 题

通过分布式数据库的学习，你的收获是什么？ 分布式数据库的内容是否对你的研究方向有帮助？有何收益？说明原因。

作答方法

写 3 到 4 点，每点写清学到的技术，以及它在自己研究方向上的用处。可以选的技术：数据分片与分配（大规模数据按场地或特征划分存储，减少传输）；分布式查询优化（选择下推、半连接，降低通信和 I/O 代价）；分布式事务和并发控制（2PC、2PL，保证多节点上的数据正确）；复制与一致性（Paxos、Raft、NWR，在可用性和一致性之间取舍）。最后用一句话概括：数据分布在多个场地以后，带来了传输代价和一致性两类问题，课程内容基本都是围绕这两点展开的。

可能出现的其他概念

分布透明性：包括分片透明（不必知道数据属于哪个片段）、分配透明（不必知道片段在哪个场地）、局部映射透明（不必知道局部的存储形式）[课件3-19]。
3PC：在 2PC 的两个阶段之间加一个“准备提交”阶段，变成投票表决、准备提交、执行三个阶段。参与者超时后可以根据其他参与者的状态决定提交或废弃，减少阻塞 [课件7-56–64]。
级联废弃：一个事务读了另一个未提交事务写的数据，后者废弃时，前者也要废弃。严格 2PL 可以避免。
时间戳法：读操作在 $ts < wts(X)$ 时被拒绝；写操作在 $ts < rts(X)$ 或 $ts < wts(X)$ 时被拒绝；被拒绝的事务换一个新时间戳重新启动。不会发生死锁，代价是重启 [课件9-38]。
CAP：一致性、可用性、分区容忍性最多同时满足两个。NWR 让用户通过参数自己选择。